

**پیاده‌سازی مدل یادگیری عمیق برای تفکیک و
برچسب‌گذاری زباله‌های قابل بازیافت مبتنی بر
شبکه‌های عصبی کانولوشنی با بهره‌گیری از کتابخانه
TensorFlow**

نگارش
ساغر اعزی

Implementation of a deep learning model for
segregation and labeling of recyclable waste
using convolutional neural networks with the
TensorFlow library

By
Saqar Aezi

Abstract

Research Aim: The objective of this project is to present a system for identifying and classifying types of waste from images, with a focus on hazardous classes, in order to assist proper separation and reduce environmental damage.

Research method: A dataset consisting of ten classes of waste was used. First, a baseline model with a simple architecture was implemented, followed by a deeper model based on Residual Blocks. To improve the model, data augmentation, class weighting, and hyperparameter search were employed. Furthermore, the FoM metric was defined as the average recall of the three hazardous classes, and the model selection and early stopping process were adjusted based on this metric. Finally, for images containing multiple objects, the Selective Search method and Non-Maximum Suppression (NMS) were used for multi-object detection.

Findings: The final model achieved a test accuracy of 0.8085 and a test FoM of 0.8423, while its test loss was reduced to 0.7917.

Conclusion: Changing the architecture to Residual Blocks and optimizing based on the FoM metric improved the model's performance in detecting hazardous waste. For further improvement, the use of more diverse data and multi-object data is suggested.

Keywords: Convolutional Neural Network, Residual Blocks, CNN.

چکیده

هدف: هدف این پروژه ارائه‌ی سیستمی برای شناسایی و طبقه‌بندی انواع زباله از روی تصویر، با تمرکز بر کلاس‌های خطرناک، به منظور کمک به تفکیک صحیح و کاهش آسیب‌های زیست‌محیطی است.

روش‌شناسی پژوهش: از یک دیتاست شامل ده کلاس زباله استفاده شد. ابتدا یک مدل پایه با معماری ساده و سپس یک مدل عمیق‌تر مبتنی بر Residual Blocks پیاده‌سازی شد و برای بهبود مدل، افزایش داده، وزن‌دهی کلاس‌ها و جستجوی هایپرپارامترها به کار گرفته شد. همچنین معیار FoM به عنوان میانگین recall سه کلاس خطرناک تعریف شد و فرایند انتخاب مدل و توقف آموزش بر اساس آن تنظیم شد. در نهایت برای تصاویر شامل چند object از روش Selective Search و Non-Maximum Suppression برای تشخیص چندگانه استفاده شد.

یافته‌ها: مدل نهایی به دقت تست ۰.۸۰۸۵ و FoM تست ۰.۸۴۲۳ دست یافت و خطای تست آن به ۰.۷۹۱۷ کاهش پیدا کرد.

نتیجه‌گیری: تغییر معماری به Residual Blocks و بهینه‌سازی بر اساس معیار FoM باعث بهبود عملکرد مدل در تشخیص زباله‌های حساس شد. برای بهبود بیشتر، استفاده از داده‌های متنوع‌تر و multi object پیشنهاد می‌شود.

کلیدواژه‌ها: طبقه‌بندی زباله، شبکه‌ی عصبی کانولوشنی، CNN، Residual Blocks.

فهرست نوشتار

۱. مقدمه	۱
۱-۱. بیان مسئله و معرفی پروژه	۱
۱-۲. مجموعه داده	۱
۱-۳. شبکه‌های عصبی و ابزارهای پیاده‌سازی	۲
۲. مدل پایه	۳
۲-۱. پیش‌پردازش	۳
۲-۲. معماری مدل	۴
۲-۲-۱. لایه‌های کانولوشنی	۴
۲-۲-۲. تابع فعال‌سازی و یادگیری مبتنی بر گرادیان	۵
۲-۲-۳. Dropout و Pooling	۸
۲-۲-۴. لایه‌های نهایی	۸
۲-۲-۵. پیش‌پردازش داده‌ها	۱۱
۲-۲-۶. تنظیمات آموزش	۱۲
۲-۲-۷. Callback‌ها	۱۴
۲-۳. جستجوی اولیه‌ی هایپرپارامترها	۱۵
۲-۴. آموزش مدل	۱۷
۳. بهبود مدل	۱۹
۳-۱. گسترش جستجوی هایپرپارامترها	۱۹
۳-۲. استفاده از Early Stopping	۲۱
۳-۳. استفاده از data augmentation	۲۳
۳-۴. Residual Blocks	۲۵
۳-۵. تغییر معماری مدل	۲۷
۳-۶. آموزش مدل	۳۰
۴. معیار FoM	۳۵
۴-۱. تعریف FoM	۳۵

۳۵	۴-۲. پیاده‌سازی FoM
۳۸	۴-۳. آموزش مدل
۴۱	۵. تشخیص چند object در تصویر
۴۱	۵-۱. بیان مسئله
۴۱	۵-۲. پیاده‌سازی
۴۷	۶. جمع‌بندی
۴۷	۶-۱. بررسی خروجی مدل روی ورودی با یک object
۴۷	۶-۲. بررسی خروجی مدل روی ورودی با multi object
۴۸	۶-۳. نتیجه
۵۰	۷. فهرست منابع

فهرست جدول‌ها

جدول ۱-۲. ترکیب هایپرپاراترهاى اولیه.....	۱۶
جدول ۱-۳. ترکیب هایپرپاراترهاى گسترده‌تر.....	۲۱
جدول ۲-۳. نتایج اجرای مدل ۱ و ۲.....	۲۳
جدول ۳-۳. نتایج اجرای مدل ۱، ۲ و ۳.....	۳۱
جدول ۴-۳. نتایج اجرای مدل ۱، ۲، ۳ و ۴.....	۳۲
جدول ۵-۳. نتایج اجرای مدل ۱، ۲، ۳، ۴ و ۵.....	۳۳
جدول ۱-۴. نتایج اجرای مدل ۱، ۲، ۳، ۴ و ۵.....	۳۸
جدول ۲-۴. نتایج اجرای تمام مدل‌ها.....	۴۰

فهرست شکل‌ها

شکل ۲-۱	تابع split_dataset	۳
شکل ۲-۲	شبکه عصبی پرسپترون	۴
شکل ۲-۳	نمونه کانولوشن	۵
شکل ۲-۴	گرادیان کاهشی	۶
شکل ۲-۵	تابع فعال‌ساز relu	۷
شکل ۲-۶	نمونه maxpooling2D	۸
شکل ۲-۷	نمونه averagepooling2D	۹
شکل ۲-۸	لایه Dense	۹
شکل ۲-۹	پیاده‌سازی مدل cnn اولیه	۱۰
شکل ۲-۱۰	پیاده‌سازی تابع train برای مدل cnn اولیه	۱۱
شکل ۲-۱۱	پیاده‌سازی callback های cnn اولیه	۱۴
شکل ۲-۱۲	نتیجه جستجوی اولیه هایپرپارامترها	۱۶
شکل ۲-۱۳	خروجی مدل پس از epoch ۴۰	۱۷
شکل ۲-۱۴	انتخاب بهترین epoch براساس val loss در اجرای epoch ۴۰	۱۷
شکل ۳-۱	پیاده‌سازی تابع hyperparameter_search	۲۰
شکل ۳-۲	ادامه پیاده‌سازی تابع hyperparameter_search	۲۱
شکل ۳-۳	ادامه پیاده‌سازی تابع hyperparameter_search	۲۲
شکل ۳-۴	خروجی مدل پس از epoch ۱۲۰	۲۲
شکل ۳-۵	پیاده‌سازی تابع train_aug	۲۴
شکل ۳-۶	پیاده‌سازی بخش افزایش داده تابع train_aug	۲۴
شکل ۳-۷	ادامه پیاده‌سازی تابع train_aug	۲۵
شکل ۳-۸	پیاده‌سازی تابع residual_block	۲۷
شکل ۳-۹	پیاده‌سازی مدل cnn جدید با residual blocks	۲۷
شکل ۳-۱۰	ادامه پیاده‌سازی مدل cnn جدید با residual blocks	۲۸
شکل ۳-۱۱	ادامه پیاده‌سازی مدل cnn جدید با residual blocks	۲۹
شکل ۳-۱۲	نتیجه جستجوی هایپرپارامترهای گسترده‌تر	۳۰
شکل ۳-۱۳	خروجی مدل پس از epoch ۱۵۰ با early stopping	۳۰

شکل ۳-۱۴	early stopping بدون epoch ۱۵۰	۳۱
شکل ۳-۱۵	early stopping بدون epoch ۲۵۰	۳۳
شکل ۴-۱	task_fom پیاده‌سازی تابع	۳۶
شکل ۴-۲	پیاده‌سازی callback سفارشی	۳۷
شکل ۴-۳	FoM تغییر callbackها برای کار با	۳۷
شکل ۴-۴	evaluate_test پیاده‌سازی تابع	۳۸
شکل ۴-۵	FoM با early stopping بدون epoch ۲۰۰	۳۸
شکل ۴-۶	FoM با early stopping بدون epoch ۳۰۰	۴۰
شکل ۵-۱	generate_regions پیاده‌سازی تابع	۴۱
شکل ۵-۲	create_box پیاده‌سازی تابع	۴۲
شکل ۵-۳	predic_rios پیاده‌سازی تابع	۴۲
شکل ۵-۴	build_objects پیاده‌سازی تابع	۴۳
شکل ۵-۵	IoU	۴۳
شکل ۵-۶	apply_nms پیاده‌سازی تابع	۴۴
شکل ۵-۷	apply_nms_per_class پیاده‌سازی تابع	۴۴
شکل ۵-۸	multi_objects_detection پیاده‌سازی تابع	۴۵
شکل ۶-۱	single object image	۴۷
شکل ۶-۲	single object image خروجی مدل با	۴۷
شکل ۶-۳	multi object image خروجی مدل با	۴۸

۱. مقدمه.

۱-۱. بیان مسئله و معرفی پروژه.

امروزه تولید زباله به طور چشمگیری افزایش یافته است. اگر این زباله‌ها به درستی تفکیک و بازیافت نشوند، خیلی زود به خاک، آب و محیط زیست آسیب می‌زنند. اکثر زباله‌ها قاطی هستند و همین باعث می‌شود بازیافتشان سخت و پرهزینه شود. بهترین راه، تفکیک درست زباله از همان ابتداست، اما انجام این کار به صورت دستی زمان‌بر است و ممکن است خطاهای زیادی رخ دهد.

ما سیستمی برای شناسایی و طبقه‌بندی نوع زباله از روی عکس ایجاد کرده‌ایم. این سیستم زباله‌ها را به ده دسته تقسیم می‌کند: باتری، پسماند زیستی، مقوا، لباس، شیشه، فلز، کاغذ، پلاستیک، کفش و زباله‌ی غیرقابل بازیافت.

۱-۲. مجموعه داده.

برای این پروژه از یک دیتاست [1] موجود در Kaggle [2] استفاده کردیم. این دیتاست شامل ده کلاس مختلف است: باتری، پسماند زیستی، مقوا، لباس، شیشه، فلز، کاغذ، پلاستیک، کفش و زباله‌ی غیرقابل بازیافت.

در این دیتاست عدم توازن مشخصی بین کلاس‌ها دیده می‌شود. این تفاوت زیاد ممکن است مدل را به کلاس‌های پرتعداد متمایل کند. برای مقابله با این مشکل، از افزایش داده و وزن‌دهی کلاس‌ها استفاده کرده‌ایم تا اثر این عدم تعادل را کاهش دهیم.

۳-۱. شبکه‌های عصبی و ابزارهای پیاده‌سازی.

ایده‌ی اصلی شبکه‌های عصبی از ساختار مغز انسان گرفته شده است. این شبکه‌ها از تعداد زیادی واحد کوچک به نام نورون تشکیل می‌شوند. هر نورون ورودی‌هایی را دریافت می‌کند، روی آن‌ها محاسبه‌ای انجام می‌دهد و نتیجه را به نورون‌های لایه‌ی بعد می‌فرستد.

برای پیاده‌سازی مدل از TensorFlow استفاده کردیم که یک کتابخانه open source برای یادگیری ماشین و یادگیری عمیق است که توسط گوگل توسعه داده شده است و ابزارهای قدرتمندی برای ساخت، آموزش و اجرای مدل‌های پیچیده، به‌ویژه در پردازش تصویر را فراهم می‌کند.

Keras نیز یک کتابخانه open source برای یادگیری عمیق است که روی TensorFlow اجرا می‌شود و فرایند ساخت و آموزش شبکه‌های عصبی را ساده می‌کند و امکان نمونه‌سازی سریع، خوانایی بالا و توسعه‌پذیری را فراهم می‌کند.

۲. مدل پایه.

۲-۱. پیش پردازش.

برای ارزیابی درست مدل داده‌ها با استفاده از تابع `split_dataset` به سه بخش مجزا تقسیم کردیم:

Train: ۰.۷ دیتاست برای آموزش مدل استفاده می‌شود.

Validation: ۰.۱۵ دیتایست برای پایش عملکرد مدل در طول آموزش و تنظیم هایپرپارامترها استفاده می‌شود.

Test: ۰.۱۵ باقی‌مانده برای ارزیابی نهایی مدل روی داده‌های دیده‌نشده استفاده می‌شود.

```
def split_dataset():
    if os.path.exists(os.path.join(split_ds, 'train')) and os.listdir(os.path.join(split_ds, 'train')) != []:
        print("dataset split already exists")
        return

    class_dirs = []
    for name in os.listdir(original_ds):
        path = os.path.join(original_ds, name)
        if os.path.isdir(path):
            class_dirs.append(name)
    class_dirs.sort()
    print(f"Found classes: {class_dirs}")

    for split in ['train', 'val', 'test']:
        for cls in class_dirs:
            os.makedirs(os.path.join(split_ds, split, cls), exist_ok=True)

    random.seed(rndm)

    for cls in class_dirs:
        src = os.path.join(original_ds, cls)
        imgs = [f for f in os.listdir(src) if f.lower().endswith(('.jpg'))]
        train_val, test = train_test_split(imgs, test_size=test_sec, random_state=rndm)
        val_sec_adj = val_sec / (train_sec + val_sec)
        train, val = train_test_split(train_val, test_size=val_sec_adj, random_state=rndm)
        # copy from org to split
        for f in train:
            shutil.copy2(os.path.join(src, f), os.path.join(split_ds, 'train', cls, f))
        for f in val:
            shutil.copy2(os.path.join(src, f), os.path.join(split_ds, 'val', cls, f))
        for f in test:
            shutil.copy2(os.path.join(src, f), os.path.join(split_ds, 'test', cls, f))
        print(f"({cls}): train={len(train)}, val={len(val)}, test={len(test)}")
    print("dataset split done\n")
```

شکل ۲-۱، تابع `split_dataset`

۲-۲. معماری مدل.

مدل اولیه یک مدل Sequential شامل ۴ بلوک اصلی است.

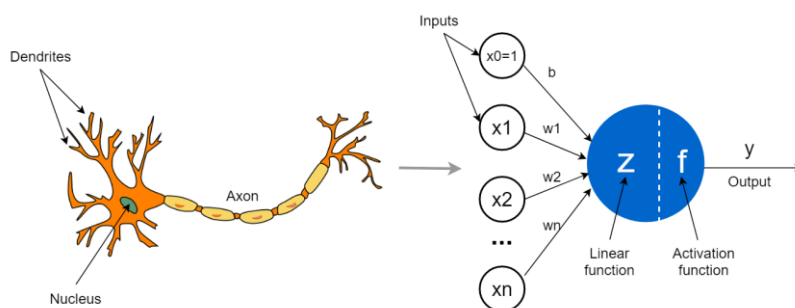
۲-۲-۱. لایه‌های کانولوشنی.

مدل Sequential در TensorFlow اجازه می‌دهد لایه‌ها را پشت سر هم و به صورت خطی قرار دهیم، به طوری که هر لایه یک ورودی و یک خروجی مشخص دارد و می‌توان لایه‌ها را اضافه یا حذف کرد. خروجی هر لایه، ورودی لایه بعدی است و مدل فقط حرکت رو به جلو دارد (حرکت بازگشتی ندارد).

ابتدا یک لایه Conv2D با ۳۲ فیلتر و اندازه کرنل ۳ اضافه می‌کنیم.

شبکه عصبی کانولوشنی، مانند سایر شبکه‌های عصبی، از نورون‌هایی با وزن‌ها و بایاس‌های قابل یادگیری تشکیل شده است. هر نورون تعدادی ورودی دریافت می‌کند، ضرب داخلی بین وزن‌ها و ورودی‌ها انجام می‌شود، با بایاس جمع می‌شود و در نهایت یک تابع فعال‌ساز به نتیجه اعمال می‌شود.

در شکل زیر یک شبکه عصبی پرسپترون که ساده‌ترین نوع شبکه با یک نورون است نشان داده شده است.

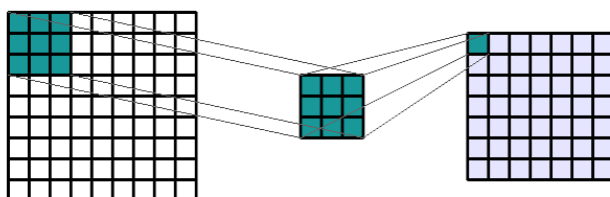


شکل ۲-۲، شبکه عصبی پرسپترون [3]

شبکه‌های عصبی معمولی با داده‌های تک‌بعدی کار می‌کنند، اما شبکه عصبی کانولوشنی

دو بعدی برای کار با تصاویر که ساختار دوبعدی دارند طراحی شده است.

در این شبکه، یک پنجره کوچک با ابعاد مشخص (کرنل) روی تصویر ورودی حرکت می‌کند. در هر موقعیت، مقادیر پیکسل‌های داخل پنجره با مقادیر کرنل ضرب شده و مجموع آنها به عنوان یک مقدار در خروجی ذخیره می‌شود. با این کار، ویژگی‌های تصویر استخراج می‌شوند، به‌طوری که در لایه‌های ابتدایی ویژگی‌های ساده و در لایه‌های عمیق‌تر ویژگی‌های پیچیده‌تر به دست می‌آیند.



شکل ۳-۲، نمونه کانولوشن [4]

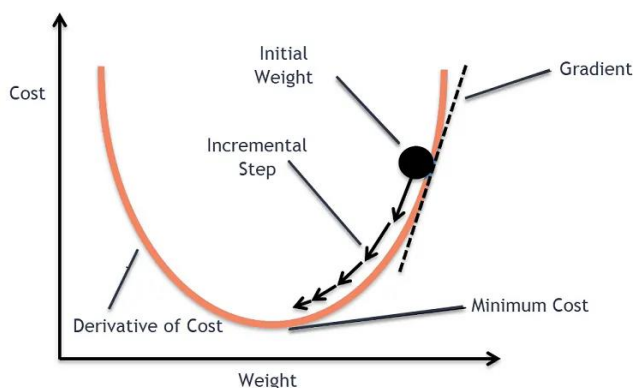
برای مثال، در گام‌های ابتدایی تصویر به بخش‌های کوچک‌تر تقسیم می‌شود و سپس در گام‌های بعدی هر بخش باز هم کوچک‌تر شده و اطلاعات جزئی‌تری استخراج می‌شود.

بنابراین یک کرنل 3×3 روی تصویر ورودی حرکت می‌کند و خروجی ایجاد می‌شود. پس برای تصویر 224×224 ، خروجی این لایه اولیه 222×222 می‌شود.

۲-۲-۲. تابع فعال‌سازی و یادگیری مبتنی بر گرادیان.

در شبکه‌های عصبی، ما یک تابع هزینه داریم که میزان خطای مدل را روی داده‌های آموزشی اندازه می‌گیرد. هدف از آموزش، یافتن مقادیری برای وزن‌ها و بایاس‌های شبکه است که این خطا را کمینه کنند. گرادیان تابع هزینه نسبت به هر پارامتر دقیقاً مشخص می‌کند که اگر آن پارامتر را اندکی افزایش دهیم، خطا چقدر تغییر خواهد کرد زیرا گرادیان تابع، نقطه‌ای را روی تابع نشان می‌دهد که تابع در آن نقطه تندترین شیب را دارد پس همواره خلاف جهت گرادیان، جهت تندترین نزول را نشان می‌دهد و از همین اصل برای تصحیح وزن‌ها استفاده می‌شود؛ هر وزن با گامی متناسب با گرادیان منفی‌اش به‌روز می‌شود

تا مدل به تدریج به سمت کمینه خطا حرکت کند.

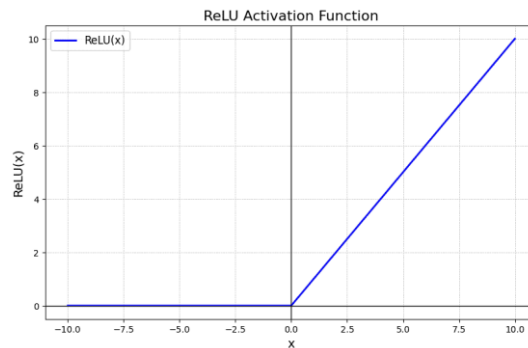


شکل ۴-۲، گرادیان کاهشی [5]

فرایند بهروزرسانی وزن‌ها با استفاده از گرادیان، گرادیان کاهشی (Gradient Descent) نام دارد. در هر گام، پارامترها در خلاف جهت گرادیان (یعنی به سمت پایین‌ترین نقطه تابع هزینه) به اندازه‌ای نرخ یادگیری جابه‌جا می‌شوند.

نرخ یادگیری (Learning Rate)، اندازه گام‌هایی را که مدل برای اصلاح وزن‌ها پس از هر مرحله آموزش برمی‌دارد، کنترل می‌کند. این هاپیرپارامتر تعیین می‌کند که وزن‌های شبکه تا چه اندازه در خلاف جهت گرادیان تابع هزینه بهروزرسانی شوند. اگر نرخ یادگیری خیلی بزرگ باشد، مدل ممکن است از کمینه هزینه عبور کند و نوسان کند یا حتی واگرا شود. اگر خیلی کوچک باشد، همگرایی به کندی پیش می‌رود و آموزش ممکن است زمان زیادی بگیرد یا در کمینه‌های محلی گرفتار شود. بنابراین، انتخاب نرخ یادگیری مناسب برای همگرایی سریع و پایدار ضروری است [6].

سپس از تابع فعال‌ساز ReLu استفاده کردیم که زمانی که مقدار ورودی منفی باشد، ReLu خروجی را صفر می‌کند. اگر مقدار ورودی مثبت یا صفر باشد، ReLu همان مقدار ورودی را برمی‌گرداند. بازه خروجی این تابع بین صفر تا بی‌نهایت است.



شکل ۵-۲، تابع فعال‌ساز relu [7]

یکی از مهم‌ترین مزایای ReLu کمک به کاهش مشکل محوشدگی گرادیان (vanishing gradient) است. این مشکل زمانی رخ می‌دهد که گرادیان مورد استفاده برای به‌روزرسانی وزن‌های شبکه، در حین انتشار رو به عقب در لایه‌های عمیق به تدریج بسیار کوچک شده و عملاً محو می‌شود. در نتیجه وزن‌ها به درستی به‌روز نمی‌شوند و فرایند یادگیری کند یا متوقف می‌شود [8].

تابع ReLu به صورت زیر تعریف می‌شود:

$$f(x) = \max(0, x) \quad 1$$

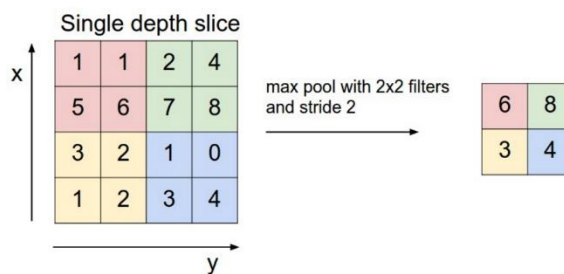
مشتق این تابع نیز به صورت زیر تعریف می‌شود:

$$f'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases} \quad 2$$

از آنجا که مشتق این تابع برای ورودی‌های مثبت همواره برابر با ۱ است، گرادیان در مسیرهای فعال شبکه بدون کاهش منتشر می‌شود و مشکل محوشدگی گرادیان را برطرف می‌کند. با این حال، یک نقطه‌ضعف در ReLu وجود دارد که به Dead State معروف است یعنی اگر نورونی در ناحیه‌ی منفی قرار گیرد و ورودی آن برای همیشه منفی بماند، خروجی و گرادیان آن صفر شده و نورون برای همیشه از فرایند یادگیری خارج می‌شود. این وضعیت معمولاً به دلیل نرخ یادگیری بیش‌ازحد بالا یا مقداردهی نامناسب بایاس رخ می‌دهد [9].

۲-۲-۳. لایه‌های Pooling و Dropout.

پس از اولین لایه Conv2D، یک لایه Conv2D دیگر با همان ۳۲ فیلتر و کرنل ۳ اضافه می‌کنیم تا شبکه بتواند الگوهای پیچیده‌تری را بدون افزایش شدید پارامترها یاد بگیرد. سپس یک لایه MaxPooling2D با کرنل ۲ می‌آید که با انتخاب بزرگ‌ترین مقدار در هر کرنل، ابعاد فضایی را نصف می‌کند.



شکل ۶-۲، نمونه maxpooling2D [10]

سپس یک Dropout با نرخ ۰.۲۵ قرار دارد که در هر مرحله آموزش ۲۵ درصد از نورون‌ها را غیرفعال می‌کند تا از بیش‌برازش جلوگیری شود. با غیرفعال‌سازی تصادفی برخی نورون‌ها، مدل برای تولید خروجی به نورون خاصی وابسته نمی‌شود.

بلوک دوم از دو لایه Conv2D با ۶۴ فیلتر و کرنل ۳ تشکیل شده است. افزایش تعداد فیلترها به شبکه کمک می‌کند ویژگی‌های سطح بالاتری را استخراج کند. سپس یک MaxPooling2D ابعاد را نصف کرده و یک Dropout اعمال می‌شود.

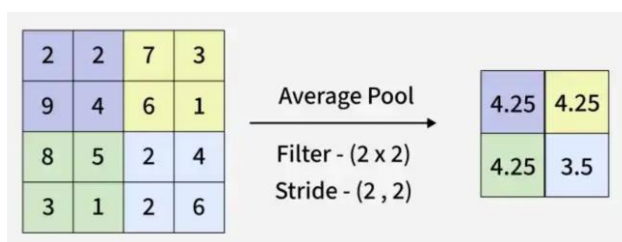
بلوک سوم شامل یک لایه Conv2D با ۱۲۸ فیلتر و سپس MaxPooling و Dropout است. در این عمق، یک لایه کانولوشنی کافی است و از پیچیدگی اضافی جلوگیری می‌کند.

۲-۲-۴. لایه‌های نهایی.

در انتهای بلوک‌های کانولوشنی، با حجمی از نقشه‌های ویژگی به ابعاد بزرگ مواجه هستیم. روش مرسوم آن است که این حجم را با لایه‌ی Flatten به یک بردار تخت با $\text{height} \times \text{width} \times \text{channel}$ عنصر تبدیل کرده و سپس یک یا چند لایه‌ی fully connected

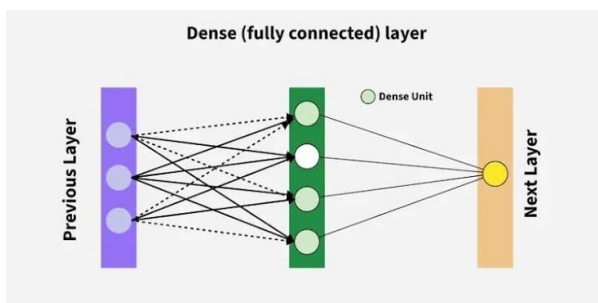
برای یادگیری الگوها از این بردار عظیم آموزش دهیم. اما این رویکرد دو مشکل اساسی دارد؛ افزایش شدید تعداد پارامترها و در نتیجه خطر بیش‌برازش و وابستگی به ابعاد فضایی ثابت که انعطاف‌پذیری شبکه را کاهش می‌دهد.

پس به جای Flatten از GlobalAveragePooling2D استفاده می‌کنیم. این لایه، برای هر یک از ۱۲۸ کانال، میانگین تمام مقادیر آن کانال روی ابعاد را محاسبه می‌کند و نتیجه یک بردار فشرده به طول ۱۲۸ است. پس به جای بردار ۸۰۰۰۰ عنصری (۲۵×۲۵×۱۲۸)، تنها ۱۲۸ عدد به لایه بعدی منتقل می‌شود [11].



شکل ۲-۲، نمونه averagepooling2D [12]

سپس یک لایه fully connected یا Dense با ۱۲۸ نورون و تابع فعال‌سازی ReLu اضافه می‌شود تا ترکیب ویژگی‌های استخراج‌شده را یاد بگیرد. سپس یک Dropout قوی با نرخ ۰.۵ قرار می‌گیرد. در نهایت، لایه خروجی Dense با تعداد نورون برابر با تعداد کلاس‌ها و تابع softmax توزیع احتمال روی کلاس‌ها را تولید می‌کند.



شکل ۲-۸، لایه Dense [13]

توابع فعال‌سازی اهمیت بالایی در شبکه‌های عصبی دارند زیرا با وارد کردن غیرخطی بودن

به شبکه‌های عصبی، امکان یادگیری الگوهای پیچیده را فراهم می‌کنند. تابع Softmax یکی از این توابع است که یک بردار از اعداد دلخواه را به یک توزیع احتمال تبدیل می‌کند، طوری که هر مقدار نشان‌دهنده‌ی احتمال تعلق ورودی به یک کلاس مشخص است. این تابع برای مسائل طبقه‌بندی چندکلاسه استفاده می‌شود. پس هر مقدار خروجی بین صفر و یک قرار دارد و مجموع تمامی مقادیر خروجی برابر یک می‌شود.

پس لایه‌ی خروجی با دریافت ویژگی‌های استخراج‌شده از لایه‌های پیشین، برای هر یک از ۱۰ کلاس یک امتیاز محاسبه می‌کند. سپس تابع Softmax این امتیازها را به احتمال تبدیل کرده و کلاسی که بیشترین احتمال را داشته باشد، به‌عنوان پیش‌بینی نهایی انتخاب می‌شود.

شکل زیر پیاده‌سازی مدل را نشان می‌دهد.

```
119 def cnn(input_shape=(224, 224, 3), num_classes=10):
120     model = models.Sequential([
121         layers.Conv2D(32,3, activation='relu', input_shape=input_shape),
122         layers.Conv2D(32,3, activation='relu'),
123         layers.MaxPooling2D(2),
124         layers.Dropout(0.25),
125
126         layers.Conv2D(64,3, activation='relu'),
127         layers.Conv2D(64,3, activation='relu'),
128         layers.MaxPooling2D(2),
129         layers.Dropout(0.25),
130
131         layers.Conv2D(128,3, activation='relu'),
132         layers.MaxPooling2D(2),
133         layers.Dropout(0.25),
134
135         layers.GlobalAveragePooling2D(),
136         layers.Dense(128, activation='relu'),
137         layers.Dropout(0.5),
138         layers.Dense(num_classes, activation='softmax')
139     ])
140     return model
```

شکل ۹-۲، پیاده‌سازی مدل cnn/اولیه

برای اجرا کردن مدل روی داده از کلاس زیر استفاده کردیم.

```

308 def train(hparams, train_dir):
309     epoch = 250
310     # early_stopping_p = 12
311     lr_p = 3
312     lr_value = 0.5
313     min_lr = 1e-7
314     lr = hparams['lr']
315     bs = hparams['batch_size']
316     drop = hparams['dropout']
317     filters = hparams['filters_base']
318
319     train_datagen = ImageDataGenerator(rescale=1./255,
320                                       rotation_range=20,
321                                       width_shift_range=0.2,
322                                       height_shift_range=0.2,
323                                       shear_range=0.2,
324                                       zoom_range=0.2,
325                                       horizontal_flip=True,
326                                       fill_mode='nearest')
327
328     val_datagen = ImageDataGenerator(rescale=1./255)

```

شکل ۱۰-۲، پیاده‌سازی تابع train برای مدل cnn اولیه

۵-۲-۲. پیش‌پردازش داده‌ها.

برای ساده‌تر شدن کار با تصاویر، از کلاس ImageDataGenerator استفاده کردیم. این کلاس به‌صورت خودکار تصاویر را می‌خواند، آن‌ها را در دسته‌های کوچک (batch) آماده می‌کند و به مدل می‌دهد. پس دیگر لازم نیست خودمان تک‌تک عکس‌ها را باز کنیم، اندازه‌شان را تغییر دهیم یا نرمال‌سازی کنیم. علاوه بر این، این کلاس می‌تواند تصاویر را به‌صورت تصادفی تغییر دهد تا مدل بهتر یاد بگیرد. مثلاً می‌تواند عکس‌ها را کمی بچرخاند، به چپ و راست جابه‌جا کند، بزرگ‌تر یا کوچک‌تر کند چون وقتی مدل با تصاویر متنوع‌تری آموزش ببیند، روی تصاویر جدید بهتر عمل می‌کند و کمتر دچار بیش‌برازش می‌شود.

نکته مهم این است که افزایش داده به‌صورت آنلاین انجام می‌شود یعنی تصاویر تغییر یافته جایی روی دیسک ذخیره نمی‌شوند، بلکه همان لحظه آموزش ساخته می‌شوند و بعد از استفاده از بین می‌روند. این کار هم فضای ذخیره‌سازی اضافه‌ای اشغال نمی‌کند و هم باعث می‌شود مدل در هر epoch با نسخه‌های کمی متفاوتی از تصاویر آموزش ببیند.

نرمال‌سازی ورودی یکی از گام‌های پیش‌پردازش کلیدی در آموزش شبکه‌های عصبی

است. تصاویر خام معمولاً دارای مقادیر پیکسلی بین ۰ تا ۲۵۵ هستند. اگر این مقادیر بدون تغییر به شبکه داده شوند، اختلاف مقیاس زیاد آنها باعث می‌شود وزن‌های شبکه به‌درستی به‌روزرسانی نشوند و فرایند یادگیری ناپایدار شود. با نرمال‌سازی، همه ویژگی‌ها در یک مقیاس مشابه قرار می‌گیرند.

برای داده‌های آموزشی از ImageDataGenerator برای نرمال‌سازی پیکسل‌ها و Data Augmentation استفاده شده است زیرا این تغییرات به‌صورت آنلاین و در حین آموزش روی هر دسته اعمال می‌شوند تا مدل با نسخه‌های متنوع‌تری از تصاویر مواجه شود و در نتیجه نسبت به تغییرات ظاهری مقاوم‌تر شده و کمتر دچار بیش‌برازش شود. اما برای داده‌های اعتبارسنجی تنها از نرمال‌سازی پیکسل‌ها استفاده شده و هیچ تغییر تصادفی دیگری اعمال نمی‌گردد. دلیل این کار آن است که مجموعه اعتبارسنجی باید نماینده‌ی داده‌های واقعی و بدون دستکاری باشد تا معیار ارزیابی مدل به‌صورت درست محاسبه شود. اگر افزایش داده روی داده‌های اعتبارسنجی نیز اعمال می‌شد، عملکرد مدل روی تصاویر تغییر یافته سنجیده می‌شد و نتیجه ارزیابی قابل اتکا نبود. بنابراین، افزایش داده فقط برای آموزش به کار می‌رود و اعتبارسنجی با داده‌های خام و نرمال شده انجام می‌شود.

۶-۲-۲. تنظیمات آموزش.

برای آماده‌سازی مدل پیش از شروع آموزش، از متد compile استفاده می‌کنیم که تنظیمات لازم برای فرایند بهینه‌سازی را مشخص می‌کند و سه مؤلفه اصلی دارد:

Optimizer: الگوریتم بهینه‌سازی که نحوه به‌روزرسانی وزن‌ها را تعیین می‌کند.

loss: تابع هزینه که اختلاف بین پیش‌بینی مدل و مقادیر واقعی را اندازه می‌گیرد.

metrics: معیارهای ارزیابی که در طول آموزش و اعتبارسنجی محاسبه می‌شوند. از accuracy استفاده کردیم که نسبت پیش‌بینی‌های صحیح به کل نمونه‌ها را نشان می‌دهد و دیدی سریع از عملکرد مدل فراهم می‌کند. حال با فراخوانی compile، مدل برای اجرای فرایند آموزش آماده می‌شود.

برای بهینه‌سازی وزن‌های مدل، از الگوریتم Adam استفاده کردیم که مخفف Adaptive

Moment Estimation است و یکی از محبوب‌ترین بهینه‌سازها در یادگیری عمیق است. این بهینه‌ساز ویژگی‌های دو بهینه‌ساز RMSProp (برای تغییر دینامیک نرخ یادگیری) و Momentum (برای هموارسازی) را ترکیب می‌کند.

RMSProp که مخفف Root Mean Square Propagation است، بهینه‌سازی است که برای حل برخی از مشکلات مربوط به کاهش سریع نرخ یادگیری به کار می‌رود. RMSProp با تنظیم نرخ یادگیری هر پارامتر بر اساس میانگین مربعات گرادیان‌های اخیر، از افت شدید و دائمی نرخ یادگیری در طول زمان جلوگیری می‌کند.

بهینه‌ساز Momentum یا تکانه با اضافه کردن بخشی از حرکت به گرادیان‌ها، به‌روزرسانی‌ها را به‌گونه‌ای هدایت می‌کند که هموار باشند.

به این ترتیب، Adam هم از مزایای Momentum (همگرایی سریع‌تر و کاهش نوسان) و هم از مزایای RMSProp (تنظیم تطبیقی نرخ یادگیری) بهره می‌برد و برای مسائل پیچیده طبقه‌بندی انتخاب مناسبی است.

نرخ یادگیری از مرحله جستجوی هایپرپارامترها تعیین شده و به Adam ارسال می‌شود. برای محاسبه‌ی loss از تابع هزینه‌ی Categorical Cross-Entropy استفاده شده. این تابع میزان اختلاف بین توزیع احتمال پیش‌بینی‌شده توسط مدل (خروجی softmax) و برچسب‌های واقعی را اندازه می‌گیرد و در طول آموزش، وزن‌ها را به‌گونه‌ای به‌روزرسانی می‌کند که این خطا کمینه شود و در نتیجه دقت طبقه‌بندی افزایش یابد.

Categorical Cross-Entropy میانگین خطای Cross-Entropy را روی تمام نمونه‌های داده محاسبه می‌کند. فرمول ریاضی آن به این صورت است:

$$L(y, \hat{y}) = - \sum_{i=1}^c y_i \log(\hat{y}_i) \quad 3$$

که y_i نشان‌دهنده‌ی برچسب واقعی کلاس i و $\hat{y}_i$ نشان‌دهنده‌ی پیش‌بینی مدل برای کلاس i است و c تعداد کلاس‌ها است.

این تابع برای هر نمونه، لگاریتم احتمال پیش‌بینی‌شده برای کلاس درست را در نظر

می‌گیرد. اگر مدل به کلاس صحیح احتمال بالایی اختصاص دهد، مقدار خطا کوچک می‌شود؛ اما اگر احتمال کمی برای کلاس درست در نظر بگیرد، جریمه‌ی سنگینی به آن اعمال می‌گردد.

۷-۲-۲. Callback ها.

Callback ها ابزارهایی هستند که در طول آموزش، در نقاط مشخصی مانند پایان هر epoch فراخوانی می‌شوند و به ما امکان می‌دهند تا عملیات مختلفی مانند ذخیره‌سازی مدل، تنظیم نرخ یادگیری یا توقف زودهنگام آموزش را به‌صورت خودکار انجام دهیم. برای ذخیره مدل یا وزن‌ها در فواصل مشخص در طول آموزش، از callback ای به نام ModelCheckpoint استفاده می‌کنیم که در پایان هر epoch مدل کامل را ذخیره می‌کند یعنی برای هر epoch یک فایل جداگانه با شماره همان epoch ساخته می‌شود پس کل مدل شامل معماری، وزن‌ها و تنظیمات بهینه‌ساز ذخیره شود، نه فقط وزن‌ها. این قابلیت به ما اجازه می‌دهد اگر فرایند آموزش به هر دلیلی متوقف شد، بتوانیم از آخرین epoch ذخیره‌شده آموزش را ادامه دهیم و نیازی به شروع مجدد از صفر نباشد. علاوه بر ذخیره‌سازی در هر epoch، یک ModelCheckpoint دیگر تعریف کردیم که تنها بهترین مدل را بر اساس کمترین مقدار val_loss ذخیره می‌کند. بخش `monitor=val_loss` مشخص می‌کند که معیار ارزیابی، خطای اعتبارسنجی است و `mode=min` یعنی هدف، کمینه کردن این مقدار است.

برای جلوگیری از گیر کردن مدل در کمینه‌های محلی و بهبود دقت در مراحل پایانی آموزش، از `ReduceLROnPlateau` استفاده کردیم. این callback مقدار `val_loss` را در پایان هر epoch بررسی می‌کند. اگر برای چند epoch متوالی هیچ بهبودی در `val_loss` مشاهده نشود، نرخ یادگیری به‌صورت خودکار با ضریب مشخصی کاهش می‌یابد [14]. به این ترتیب، گام‌های به‌روزرسانی وزن‌ها کوچک‌تر می‌شوند و مدل می‌تواند با دقت بیشتری به سمت کمینه تابع هزینه حرکت کند. `min_lr` نیز حداقل نرخ یادگیری را تعیین می‌کند تا از کاهش بیش‌از حد و توقف کامل یادگیری جلوگیری شود.

```

355 ~ callbacks = {
356     tf.keras.callbacks.ModelCheckpoint(filepath=os.path.join(ckpt_dir, 'model_epoch_{epoch:02d}.h5'), save_weights_only=False, save_freq='epoch', verbose=1),
357     tf.keras.callbacks.ModelCheckpoint(filepath=best_model_path, monitor='val_loss', save_best_only=True, mode='min', verbose=1),
358     # tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=early_stopping_p, restore_best_weights=True, verbose=1),
359     tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=lr_value, patience=lr_p, min_lr=min_lr, verbose=1)
360 }

```

شکل ۱۱-۲، پیاده‌سازی callback های cnn/اولیه

برای اجرای فرایند آموزش نهایی مدل، از متد `model.fit` استفاده کردیم. این متد مدل را برای تعداد مشخصی epoch روی داده‌های آموزشی آموزش می‌دهد. در طول هر epoch، مدل وزن‌ها و بایاس‌های خود را به‌روزرسانی می‌کند تا خطای خروجی کاهش یابد.

در هر epoch، فرایند آموزش شامل چهار مرحله اصلی است. ابتدا Forward Pass که تصاویر ورودی از لایه‌های شبکه عبور داده می‌شوند و مدل برای هر نمونه یک پیش‌بینی تولید می‌کند. سپس Loss Calculation که پیش‌بینی‌های مدل با برچسب‌های واقعی مقایسه شده و مقدار خطا با استفاده از تابع هزینه به دست می‌آید. سپس Backward Pass که گرادیان‌های خطا نسبت به تمام وزن‌ها و بایاس‌های شبکه با الگوریتم پس‌انتشار (Backpropagation) محاسبه می‌شوند و در آخر Weight Update که وزن‌ها و گرادیان‌های محاسبه‌شده به‌روزرسانی می‌شوند تا خطا در epoch بعدی کاهش یابد. خروجی `model.fit` یک History است که مقادیر loss و accuracy را برای هر epoch روی داده‌های آموزشی و اعتبارسنجی ذخیره می‌کند [15].

سپس مدل را ذخیره می‌کنیم.

۲-۳. جستجوی اولیه‌ی هایپرپارامترها.

قبل از اجرای مدل اصلی، برای اینکه نتیجه بهتری بگیریم، ابتدا یک مرحله‌ی جستجوی هایپرپارامتر انجام دادیم. نکته‌ی مهم این است که این جستجو با تعداد epoch کمتر انجام شده یعنی در مرحله‌ی جستجو برای صرفه‌جویی در زمان و منابع، هر ترکیب را فقط ۱۵ epoch آموزش می‌دهیم و بهترین آن‌ها را بر اساس عملکرد روی مجموعه اعتبارسنجی انتخاب می‌کنیم. این کار به ما امکان می‌دهد تا بدون صرف هزینه‌ی کامل آموزش، تنظیمات مناسب را پیدا کنیم.

هدف از این جستجو یافتن مناسب‌ترین مقادیر برای دو هایپرپارامتر مهم یعنی نرخ

یادگیری و Batch Size است. این دو پارامتر تأثیر مستقیمی بر سرعت همگرایی، پایداری آموزش و دقت نهایی مدل دارند. اگر نرخ یادگیری بیش از حد بزرگ باشد، مدل ممکن است اصلاً یاد نگیرد یا واگرا شود؛ اگر خیلی کوچک باشد، آموزش بسیار کند پیش می‌رود پس، با صرف هزینه‌ی محاسباتی نسبتاً کم در این مرحله، می‌توانیم بهترین ترکیب را شناسایی کنیم و سپس مدل نهایی را با همان تنظیمات به مدت طولانی‌تر آموزش دهیم.

برای این کار، سه ترکیب مختلف انتخاب شدند:

جدول ۱-۲. ترکیب های پارامترهای اولیه

ردیف	Learning rate	Batch size
۱	۰.۰۰۰۵	۱۶
۲	۰.۰۰۱	۳۲
۳	۰.۰۱	۶۴

سپس ترکیب در epoch ۱۵ روی داده‌های آموزشی آموزش داده شد و عملکرد آن روی داده‌های اعتبارسنجی با معیار val_loss ارزیابی شد.

نتایج جستجو به این صورت بود:

```
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
134/134 - 496s - 4s/step - accuracy: 0.1544 - loss: 2.2263 - val_accuracy: 0.1541 - val_loss: 2.2263 - learning_rate: 0.0012
Epoch 14/15
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy.
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
134/134 - 496s - 4s/step - accuracy: 0.1544 - loss: 2.2261 - val_accuracy: 0.1541 - val_loss: 2.2263 - learning_rate: 6.2500e-04
Epoch 15/15
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy.
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
134/134 - 491s - 4s/step - accuracy: 0.1544 - loss: 2.2262 - val_accuracy: 0.1541 - val_loss: 2.2263 - learning_rate: 6.2500e-04
best val_loss = 2.2263

SEARCH RESULTS
lr=0.00050, bs=16 val_loss=1.3985 - BEST
lr=0.00100, bs=32 val_loss=1.4172
lr=0.01000, bs=64 val_loss=2.2263
Best hyperparameters saved to best_hyperparams.txt
```

شکل ۱۲-۲، نتیجه جستجوی اولیه های پارامترها

ترکیب اول بهترین عملکرد را از نظر کمترین خطای اعتبارسنجی نشان داد. خطای اعتبارسنجی به‌صورت پیوسته کاهش یافت و دقت اعتبارسنجی نیز بهبود یافت. روند یادگیری در این ترکیب بسیار پایدار بود و کاهش خطا در تمام epoch ها ادامه داشت. به

همین دلیل، این ترکیب به عنوان بهترین انتخاب شد.

برای اینکه متوجه شویم مدل چقدر خوب عمل می کند، از تابع `evaluate_test` استفاده کردیم. این تابع مدل آموزش دیده را دریافت می کند و ابتدا برای پیش پردازش تصاویر تست، از `ImageDataGenerator` برای `rescale` استفاده کردیم تا فقط نرمال سازی پیکسل ها انجام شود و هیچ افزایش داده ای روی داده های تست اعمال نشود چون هدف، سنجش عملکرد مدل روی داده های واقعی و بدون تغییر است. سپس برای بارگذاری تصاویر تست به صورت دسته ای، از متد `flow_from_directory` استفاده کردیم که تصاویر را به ابعاد مشخص شده تغییر اندازه می دهد، `batch_size` مشخص می کند که در هر دسته ۳۲ تصویر قرار بگیرد.

سپس از تابع `model.evaluate` استفاده کردیم. این تابع مدل را روی یک مجموعه داده مشخص ارزیابی می کند و مقدار خطا و `accuracy` را برمی گرداند. این ارزیابی روی داده های تست انجام می شود که مدل در طول آموزش هیچ گاه آن ها را مشاهده نکرده است. خروجی این تابع شامل مقدار خطا و دقت مدل روی داده های تست است.

۴-۲. آموزش مدل.

سپس مدل را با `epoch ۴۰` اجرا کردیم با هایپر پارامترهای مشخص شده.

خروجی به صورت زیر است.

```
536/536 — 0s 751ms/step - accuracy: 0.6443 - loss: 1.0428
Epoch 40: saving model to checkpoints\model_epoch_40.h5
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy.
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Epoch 40: finished saving model to checkpoints\model_epoch_40.h5
Epoch 40: val_loss improved from 1.05571 to 1.05060, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy.
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Epoch 40: finished saving model to best_model.h5
536/536 — 419s 782ms/step - accuracy: 0.6404 - loss: 1.0546 - val_accuracy: 0.6582 - val_loss: 1.0506 - learning_rate: 1.2500e-04
Restoring model weights from the end of the best epoch: 40.
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy.
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Best model saved to best_model.h5
Evaluating on test set
Found 1843 images belonging to 10 classes.
58/58 — 16s 281ms/step - accuracy: 0.6511 - loss: 1.0815
Test accuracy: 0.6511 | Test loss: 1.0815
```

شکل ۱۳-۲، خروجی مدل پس از `epoch ۴۰`

همانطور که مشخص است بر اساس کمترین خطای اعتبارسنجی، `epoch` شماره ۴۰ بهترین عملکرد را داشت با `val_loss = ۱.۰۵۰۶` و `val_accuracy = ۰.۶۵۸۲`.

```
Epoch 40: finished saving model to checkpoints\model_epoch_40.h5
Epoch 40: val_loss improved from 1.05571 to 1.05060, saving model to best_model.h5
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Epoch 40: finished saving model to best_model.h5
```

شکل ۱۴-۲، انتخاب بهترین epoch براساس val loss در اجرای epoch ۴۰

نتیجه اجرای مدل روی داده‌های تست:

دقت تست: ۰.۶۵۱۱

خطای تست: ۱.۰۸۱۵

مدل در حال حاضر می‌تواند عملکرد بهتری داشته باشد. برای بهتر شدن مدل، کارهای زیر را انجام دادیم.

۳. بهبود مدل.

۳-۱. گسترش جستجوی هایپرپارامترها.

در مراحل اولیه، جستجو فقط روی دو هایپرپارامتر نرخ یادگیری و `batch_size` انجام شد؛ اما برای بهبود بیشتر عملکرد، لازم بود هایپرپارامترهای مهم دیگری نیز بررسی شوند. به همین دلیل، در این مرحله چهار هایپرپارامتر کلیدی به صورت همزمان مورد آزمایش قرار گرفتند: نرخ یادگیری، `batch_size`، نرخ Dropout و `filters_base`.

هر یک از این هایپرپارامترها نقش مهمی در کیفیت یادگیری و تعمیم‌پذیری مدل دارند. نرخ یادگیری سرعت به‌روزرسانی وزن‌ها را کنترل می‌کند. مقدار خیلی زیاد باعث ناپایداری و واگرایی می‌شود و مقدار خیلی کم، آموزش را کند می‌کند. `batch_size` تعداد نمونه‌هایی که در هر گام به مدل داده می‌شود را مشخص می‌کند. دسته‌های کوچک‌تر نویز گرادیان را افزایش می‌دهند ولی می‌توانند به فرار از کمینه‌های محلی کمک کنند؛ دسته‌های بزرگ‌تر برآورد گرادیان پایدارتری دارند اما حافظه بیشتری مصرف می‌کنند. نرخ Dropout درصدی از نورون‌ها را در طول آموزش به صورت تصادفی غیرفعال می‌کند تا از بیش‌برازش جلوگیری شود و `filters_base` تعداد فیلترهای اولین بلوک کانولوشنی را تعیین می‌کند و مستقیماً بر ظرفیت و پیچیدگی مدل اثر می‌گذارد. افزایش این مقدار قدرت یادگیری مدل را بالا می‌برد اما خطر بیش‌برازش و هزینه محاسباتی را نیز زیاد می‌کند.

تابع `hyperparameter_search` شامل پنج ترکیب مختلف از این چهار هایپرپارامتر است که به صورت یک لیست از دیکشنری‌ها تعریف شده‌اند.

```

179 ✓ def hyperparameter_search(train_dir):
180 ✓     search_data = [
181         {'lr': 0.0005, 'batch_size': 16, 'dropout': 0.3, 'filters_base': 32},
182         {'lr': 0.001, 'batch_size': 32, 'dropout': 0.5, 'filters_base': 32},
183         {'lr': 0.01, 'batch_size': 64, 'dropout': 0.4, 'filters_base': 48},
184         {'lr': 0.001, 'batch_size': 16, 'dropout': 0.3, 'filters_base': 48},
185         {'lr': 0.0005, 'batch_size': 64, 'dropout': 0.5, 'filters_base': 32},
186     ]
187     search_epoch = 12
188     search_p = 5
189     print("\nhyperparameter search")
190
191     completed = set()
192     best_val_loss = float('inf')
193     best_hparams = None
194     results = []
195
196 ✓     if os.path.exists(search_progress_file):
197 ✓         with open(search_progress_file, 'r') as f:
198             progress = json.load(f)
199
200         saved_data = progress.get('hparamss', [])
201 ✓         if len(saved_data) == len(search_data) and saved_data == search_data:
202             completed = set(progress.get('completed', []))
203             best_val_loss = progress.get('best_val_loss', float('inf'))
204             best_hparams = progress.get('best_hparams', None)
205             results = progress.get('results', []) # (conf, val loss)
206             print(f"resuming from {len(completed)}.")
207 ✓         else:
208             print("start from scratch")
209
210 ✓     train_datagen = ImageDataGenerator(rescale=1./255,
211                                         rotation_range=20,
212                                         width_shift_range=0.2,
213                                         height_shift_range=0.2,
214                                         shear_range=0.2,
215                                         zoom_range=0.2,
216                                         horizontal_flip=True,
217                                         fill_mode='nearest')
218
219     val_datagen = ImageDataGenerator(rescale=1./255)

```

شکل ۱-۳، پیاده‌سازی تابع hyperparameter_search

در این جستجو، هر ترکیب به مدت ۱۲ epoch آموزش داده می‌شود و ترکیب چهارم به‌عنوان بهترین ترکیب انتخاب شد.

```

221 for i, hparams in enumerate(search_data):
222     if i is completed:
223         print(f"skipping hparams {i+1}/{len(search_data)}")
224         continue
225
226     lr = hparams['lr']
227     bs = hparams['batch_size']
228     drop = hparams['dropout']
229     filters = hparams['filters_base']
230
231     print(f"\nhparams {i+1}/{len(search_data)}: {hparams}")
232
233     train_gen = train_datagen.flow_from_directory(train_dir, target_size=(img_h, img_w), batch_size=bs, class_mode='categorical', shuffle=True)
234     val_gen = val_datagen.flow_from_directory(os.path.join(split_ds, 'val'), target_size=(img_h, img_w), batch_size=bs, class_mode='categorical', shuffle=False)
235
236     model = cnn(
237         input_shape=(img_h, img_w, 3),
238         num_classes=len(class_name),
239         filters_base=filters,
240         dropout_rate=drop
241     )
242     model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=lr), loss='categorical_crossentropy', metrics=['accuracy'])
243
244     temp_path = f'temp_search_{i}.h5'
245     callbacks = [
246         tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=search.p, restore_best_weights=True, verbose=0),
247         tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, verbose=0),
248         tf.keras.callbacks.ModelCheckpoint(temp_path, monitor='val_loss', save_best_only=True, verbose=0)
249     ]

```

شکل ۳-۲، داده پیاده سازی تابع hyperparameter_search

جدول ۳-۱. ترکیب های پیرامترهای گسترده تر

ردیف	Batch size	Learning rate	dropout	Filter base
۱	۱۶	۰.۰۰۰۵	۰.۳	۳۲
۲	۳۲	۰.۰۰۱	۰.۵	۳۲
۳	۶۴	۰.۰۱	۰.۴	۴۸
۴	۱۶	۰.۰۰۱	۰.۳	۴۸
۵	۶۴	۰.۰۰۰۵	۰.۵	۳۲

۳-۲. استفاده از Early Stopping.

علاوه بر گسترده تر کردن جستجوی های پیرامترها، از callback مربوط به Early Stopping نیز استفاده می کنیم که در صورت عدم بهبود معیار تحت نظارت آموزش را متوقف می کند. حلقه ی آموزش در پایان هر epoch بررسی می کند که آیا مقدار مورد نظر دیگر کاهش نمی یابد یا خیر؛ و در صورت تعیین patience، آن را نیز لحاظ می کند. به محض اینکه تشخیص دهد معیار دیگر بهبود پیدا نمی کند، فرایند آموزش خاتمه می یابد.

```

251 hist = model.fit(train_gen, validation_data=val_gen, epochs=search_epoch, callbacks=callbacks, verbose=2)
252 val_loss = min(hist.history['val_loss'])
253 results.append((hparams, val_loss))
254 print(f"best val_loss = {val_loss:.4f}")
255
256 if val_loss < best_val_loss:
257     best_val_loss = val_loss
258     best_hparams = hparams
259     model.save('best_search_model.h5')
260
261 if os.path.exists(temp_path):
262     os.remove(temp_path)
263
264 completed.add(1)
265
266 progress = {
267     'completed': list(completed),
268     'best_val_loss': best_val_loss,
269     'best_hparams': best_hparams,
270     'results': [(dict(c), v) for c, v in results],
271     'hparams': search_data
272 }
273
274 # add to json
275 with open(search_progress_file, 'w') as f:
276     json.dump(progress, f)
277
278 # if os.path.exists(search_progress_file):
279 #     os.remove(search_progress_file)
280
281 print("\nsearch results")
282 for hparams, vloss in sorted(results, key=lambda x: x[1]): # based on val loss (hparam, val loss)
283     marker = "best" if hparams == best_hparams else ""
284     print(f"{hparams} val_loss={vloss:.4f}{marker}")
285
286 with open(best_hp_file, 'w') as f:
287     f.write(f"best = {best_hparams}\n")
288     f.write(f"val_loss = {best_val_loss:.4f}\n")
289 print(f"\nbest hparam saved to {best_hp_file}")
290 return best_hparams

```

شکل ۳-۳، داده‌سازی تابع hyperparameter_search

سپس مدل را با ۱۲۰ epoch اجرا کردیم با هایپرپارامترهای مشخص شده ولی به خاطر وجود Early Stopping مدل در epoch شماره ۱۰۰ متوقف می‌شود.

```

evaluating on test set
Found 1843 images belonging to 10 classes.
C:\Users\test\AppData\Local\Programs\Python\Python313\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
  self._warn_if_super_not_called()
58/58 — 12s 184ms/step - accuracy: 0.8394 - loss: 0.9595
test accuracy: 0.8394, test loss: 0.9595

```

شکل ۴-۳، خروجی مدل پس از ۱۲۰ epoch

بر اساس کمترین خطای اعتبارسنجی، epoch شماره ۲۹ بهترین عملکرد ۰.۸۲۴۹ را داشت با $\text{val_loss} = 0.8249$ و $\text{val_accuracy} = 0.7987$.

نتیجه اجرای مدل روی داده‌های تست:

دقت تست: ۰.۸۳۹۴

خطای تست: ۰.۹۵۹۵

نسبت به وضعیت قبلی مدل، دقت مدل افزایش داشته و خطا نیز کاهش پیدا کرده.

جدول ۲-۳. نتایج اجرای مدل ۱ و ۲

detail		Test loss	Test acc	Val loss	Val acc	ردیف
cnn	hypram					
40 ep	Lr, batch size	۱.۰۸۱۵	۰.۶۵۱۱	۱.۰۵۰۶	۰.۶۵۸۲	۱
120 ep + early stopping	Lr, batch size, dropout, filter base	۰.۹۵۹۵	۰.۸۳۹۴	۰.۸۲۴۹	۰.۷۹۸۷	۲

مدل پیشرفت کرده است ولی برای استفاده از مدل در کاربردهای واقعی نیاز به بهبود بیشتر احساس می‌شود. دو محدودیت اصلی در مدل وجود دارد. مدل از یک شبکه کم‌عمق استفاده می‌کند با تنها ۴ لایه کانولوشنی. چنین معماری ممکن است توانایی کافی برای استخراج ویژگی‌های بسیار پیچیده را نداشته باشد و در دقت‌های بالاتر به نقطه اشباع برسد. یک شبکه عمیق، این تعداد داده ممکن است کافی نباشد و خطر بیش‌برازش را افزایش دهد.

۳-۳. استفاده از data augmentation.

برای رفع این محدودیت‌ها، دو تغییر اساسی اعمال کردیم: data augmentation و تغییر ساختار مدل.

برای افزایش تعداد و تنوع تصاویر آموزشی، تابع train_aug را تعریف کردیم. این تابع برای هر داده train، پنج نسخه‌ی جدید با تغییرات تصادفی تولید می‌کند. خروجی نهایی، پوشه‌ی train_augmented است که شامل نسخه‌های افزایش‌یافته است.

```

70 ∨ def train_aug():
71     src_train_dir = os.path.join(split_ds, 'train')
72     aug_train_dir = os.path.join(split_ds, aug_dir)
73
74 ∨     if os.path.exists(aug_train_dir) and os.listdir(aug_train_dir) != []:
75         print(f"augmented training folder already exists")
76         return aug_train_dir
77
78     print("\ntrain data augmentation")
79     print(f"creating {aug_num} new images per original in {aug_train_dir}")
80

```

شکل ۵-۳، پیاده‌سازی تابع train_aug

برای تولید تصاویر افزایش‌یافته، از کلاس ImageDataGenerator استفاده کردیم که تغییرات زیر را به‌صورت تصادفی روی هر تصویر اعمال می‌کند:

```

81     datagen = ImageDataGenerator(
82         rotation_range=20,
83         width_shift_range=0.2,
84         height_shift_range=0.2,
85         shear_range=0.2,
86         zoom_range=0.2,
87         horizontal_flip=True,
88         fill_mode='nearest'
89     )

```

شکل ۶-۳، پیاده‌سازی بخش افزایش داده تابع train_aug

برای هر تصویر، ابتدا تصویر با OpenCV بارگذاری شده و از فضای رنگی BGR به RGB تبدیل می‌شود تا با فرمت مورد انتظار TensorFlow سازگار باشد. سپس مقادیر پیکسل‌ها به float تبدیل می‌شود و تصویر به‌صورت یک دسته‌ی تک‌نمونه‌ای به متد datagen.flow می‌شود تا نسخه‌های افزایش‌یافته‌ی تصویر تولید شوند.

```

91     for cls in class_name:
92         cls_src = os.path.join(src_train_dir, cls)
93         cls_dst = os.path.join(aug_train_dir, cls)
94         os.makedirs(cls_dst, exist_ok=True)
95
96         img_files = [f for f in os.listdir(cls_src) if f.lower().endswith('.jpg')] # org imgs
97         print(f"class '{cls}': {len(img_files)} originals: {len(img_files) * aug_num} augmented + {len(img_files)} originals")
98
99         for fname in img_files:
100             img_path = os.path.join(cls_src, fname)
101             img = cv2.imread(img_path) # BGR
102             if img is None:
103                 continue
104             img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB) # BGR to RGB for tf
105             img_rgb = img_rgb.astype('float32') # float for tf
106             img_batch = np.expand_dims(img_rgb, axis=0) # reshape to (1,H,W,C) for generator
107             aug_iter = datagen.flow(img_batch, batch_size=1, # generate augmented images
108                                     save_to_dir=cls_dst,
109                                     save_prefix=f'aug_{os.path.splitext(fname)[0]}',
110                                     save_format='jpg')
111
112             for _ in range(aug_num):
113                 next(aug_iter)
114             shutil.copy2(img_path, os.path.join(cls_dst, fname)) # copy org imgs to aug
115
116         print(f"augmentation complete\n")
117     return aug_train_dir

```

شکل ۷-۳، داده‌های تولید شده برای train_aug

برای هر تصویر اصلی، پنج تصویر جدید تولید می‌شود.

۳-۴. Residual Blocks.

سپس ساختار مدل را تغییر می‌دهیم تا عمق بیشتری داشته باشد و بتواند ویژگی‌های پیچیده‌تری را استخراج کند. برای این منظور، به جای لایه‌های کانولوشنی ساده و پشت‌سرهم، از Residual Blocks استفاده کردیم.

در شبکه‌های عمیق، مشکل محوشدگی گرادیان باعث می‌شود لایه‌های ابتدایی به خوبی یاد نگیرند و افزودن لایه‌های بیشتر حتی دقت را کاهش دهد. راه حل معرفی شده در ResNet استفاده از اتصال‌های میان‌بر (Skip Connections) است. در این روش، خروجی هر بلوک به جای یادگیری مستقیم نداشت، تنها بخش باقی‌مانده را یاد می‌گیرد و ورودی بلوک مستقیماً به خروجی اضافه می‌شود. این کار جریان گرادیان را حفظ کرده و آموزش شبکه‌های عمیق‌تر را ممکن می‌سازد [16].

در لایه‌های ابتدایی مدل ResNet، پس از هر لایه کانولوشنی یک لایه Batch Normalization اضافه می‌شود. ترتیب این بخش به صورت $\text{ReLU} \leftarrow \text{MaxPooling} \leftarrow \text{Conv2D} \leftarrow \text{BatchNorm}$ است [17].

در پیاده‌سازی ما نیز تابع `residual_block` دقیقاً از همین ساختار پیروی می‌کند: ابتدا لایه `Conv2D`، سپس `BatchNormalization` و `ReLU`، سپس لایه `Conv2D` دوم و `BatchNormalization` و در نهایت جمع با ورودی و `ReLU` نهایی.

این تابع یک `Residual Block` می‌سازد. هر بلوک از دو مسیر اصلی تشکیل شده است:

- مسیر اصلی: شامل دو لایه کانولوشنی، دو لایه `BatchNormalization` و یک تابع فعال‌سازی.
- `Shortcut Connection`: که ورودی بلوک را مستقیماً به خروجی مسیر اصلی اضافه می‌کند.

در ابتدا، ورودی در متغیر `shortcut` ذخیره می‌شود. سپس لایه‌ی کانولوشنی اول، `BatchNormalization` و تابع فعال‌سازی اعمال می‌شوند. پس از آن، لایه‌ی کانولوشنی دوم و دوباره `BatchNormalization` قرار دارند. در این مرحله `ReLU` اعمال نمی‌شود؛ زیرا ابتدا باید خروجی مسیر اصلی با میان‌بر جمع شود و سپس فعال‌سازی نهایی انجام بگیرد.

اگر یکی از این دو حالت پیش بیاید، باید ابعاد مسیر میان‌بر با خروجی مسیر اصلی هماهنگ شود:

- وقتی `stride` برابر ۲ باشد، ابعاد فضایی خروجی نصف می‌شود پس میان‌بر هم باید به همان اندازه کوچک شود.

- وقتی تعداد کانال‌های ورودی با تعداد فیلترهای خروجی فرق دارد.

برای این کار، یک لایه `Conv2D` با کرنل 1×1 و همان `stride` روی `shortcut` اعمال می‌شود. کرنل 1×1 تعداد کانال‌ها را تغییر می‌دهد و `stride` ابعاد فضایی را تنظیم می‌کند. سپس `BatchNormalization` روی آن اعمال می‌شود.

خروجی مسیر اصلی و مسیر میان‌بر با یکدیگر جمع می‌شوند. این جمع همان اتصالی است که اجازه می‌دهد گرادیان به‌طور مستقیم از میان‌بر عبور کند و مشکل محوشدگی گرادیان کاهش یابد. پس از جمع، یک `ReLU` نهایی اعمال می‌شود تا خروجی بلوک

غیرخطی شود.

```

142 def residual_block(x, filters, kernel_size=3, stride=1):
143     shortcut = x
144     x = layers.Conv2D(filters, kernel_size, strides=stride, padding='same', kernel_initializer='he_normal')(x) # to work with Relu
145     x = layers.BatchNormalization()(x)
146     x = layers.Activation('relu')(x) # same as ResNet first batch norm then act
147     x = layers.Conv2D(filters, kernel_size, padding='same', kernel_initializer='he_normal')(x)
148     x = layers.BatchNormalization()(x)
149
150     if stride != 1 or shortcut.shape[-1] != filters:
151         # making them same sizes
152         shortcut = layers.Conv2D(filters, 1, strides=stride, padding='same', kernel_initializer='he_normal')(shortcut)
153         shortcut = layers.BatchNormalization()(shortcut)
154     x = layers.Add()([x, shortcut])
155     x = layers.Activation('relu')(x)
156     return x

```

شکل ۸-۳، پیاده‌سازی تابع residual_block

۵-۳. تغییر معماری مدل.

حال ساختار مدل را نیز تغییر می‌دهیم تا با residual blocks کار کند.

روشی دیگر برای ساخت مدل‌های Keras، استفاده از Functional Interface است. در این روش، لایه‌ها مانند توابعی عمل می‌کنند که یک تنسور را به عنوان ورودی می‌گیرند و یک تنسور دیگر را به عنوان خروجی برمی‌گردانند. رابط تابعی نسبت به مدل Sequential انعطاف‌پذیری بیشتری دارد؛ چون در آن محدود به چیدن خطی لایه‌ها نیستیم و می‌توانیم مدل‌هایی با چند مسیر یا چند ورودی بسازیم.

residual blocks نیز به دلیل داشتن Skip Connection، نمی‌تواند به صورت یک نمایش خطی ساده از لایه‌ها نمایش داده شود؛ پس باید از Functional Interface استفاده کنیم.

پس ورودی مدل را مشخص می‌کنیم. این لایه تصویری با ابعاد مشخص (input_shape) را به عنوان ورودی می‌گیرد و نقطه شروع مدل است.

```

158 def cnn(input_shape=(224, 224, 3), num_classes=10, filters_base=32, dropout_rate=0.5):
159     inputs = layers.Input(shape=input_shape)

```

شکل ۹-۳، پیاده‌سازی مدل cnn جدید با residual blocks

تابع cnn شامل سه بخش اصلی است:

- لایه‌های اولیه
- Residual Blocks

• لایه‌های نهایی

اولین لایه کانولوشنی از `filters_base` مشخص شده توسط جستجو هایپرپارامترها و کرنل 7×7 استفاده می‌کند. کرنل بزرگ‌تر میدان دید وسیع‌تری دارد و `stride` برابر ۲ ابعاد فضایی را به نصف کاهش می‌دهد و `padding=same` ابعاد خروجی را کنترل می‌کند.

در شبکه‌های عصبی، مقداردهی اولیه وزن‌ها نقش مهمی در پایداری و سرعت آموزش دارد. اگر وزن‌ها به‌درستی مقداردهی نشوند، ممکن است در همان آغاز آموزش با مشکلاتی مانند محوشدگی گرادیان مواجه شویم و مدل نتواند به‌خوبی یاد بگیرد. یکی از روش‌های رایج مقداردهی اولیه، He Initialization است که به‌طور خاص برای توابع فعال‌سازی ReLU طراحی شده است.

ReLU برای مقادیر منفی خروجی صفر تولید می‌کند؛ بنابراین واریانس وزن‌ها باید به‌گونه‌ای تنظیم شود که از ایجاد نوروں‌های مرده جلوگیری شده و گرادیان‌ها در لایه‌های عمیق به‌خوبی جریان پیدا کنند. روش He با مقیاس‌دهی مناسب وزن‌ها، واریانس خروجی هر لایه را در حد مطلوب نگه می‌دارد و خطر کاهش شدید گرادیان را کاهش می‌دهد.

مدل از `he_normal` استفاده می‌کند تا مقداردهی اولیه، هماهنگ با تابع فعال‌سازی ReLU باشد تا از همان ابتدای آموزش در مسیر درستی قرار گیرد و همگرایی سریع‌تر و پایدارتر شود.

سپس با استفاده از Batch Normalization خروجی کانولوشن نرمال شده و از ReLU عبور می‌کند. سپس با `MaxPooling2D` ابعاد دوباره نصف می‌شود.

```
158 def cnn(input_shape=(224, 224, 3), num_classes=10, filters_base=32, dropout_rate=0.5):
159     inputs = layers.Input(shape=input_shape)
160     x = layers.Conv2D(filters_base, 7, strides=2, padding='same', kernel_initializer='he_normal')(inputs) # make it ready for relu
161     x = layers.BatchNormalization()(x)
162     x = layers.Activation('relu')(x)
163     x = layers.MaxPooling2D(pool_size=3, strides=2, padding='same')(x)
```

شکل ۱۰-۳، ۱۵/۱۴ پیاده‌سازی مدل cnn جدید با residual blocks

Batch Normalization الگوریتمی است که به افزایش سرعت و پایداری آموزش شبکه‌های عصبی عمیق کمک می‌کند. این کار با نرمال‌سازی ورودی هر لایه انجام می‌شود به‌طوری که میانگین آن صفر و انحراف معیار آن یک شود.

سپس نش Residual Blocks به صورت پشت سرهم قرار گرفته اند: دو بلوک اول با تعداد فیلتر برابر `filters_base` و `stride` برابر ۱ اجرا می شوند؛ بنابراین ابعاد فضایی تغییری نمی کند. بلوک سوم با ۲ برابر `filters_base` و `stride` برابر ۲ اجرا می شود؛ در نتیجه ابعاد فضایی نصف شده و تعداد کانال ها دو برابر می شود. بلوک چهارم با ۲ برابر `filters_base` و `stride` برابر ۱ روی همان ابعاد ادامه می دهد. بلوک پنجم با ۴ برابر `filters_base` و `stride` برابر ۲ دوباره ابعاد فضایی را نصف کرده و تعداد کانال ها را چهار برابر مقدار پایه می کند. بلوک ششم با ۴ برابر `filters_base` و `stride` برابر ۱ روی همان ابعاد کار می کند.

در این مدل ابعاد فضایی به تدریج کاهش می یابد و در مقابل تعداد کانال ها افزایش پیدا می کند تا مدل بتواند از ویژگی های ساده به ویژگی های پیچیده برسد. در هر مرحله، بلوک اول با `stride` برابر ۲ ابعاد را نصف کرده و تعداد کانال ها را دو برابر می کند؛ بلوک دوم با `stride` برابر ۱ روی همان ابعاد به دست آوردن ویژگی ها می پردازد.

سپس با `GlobalAveragePooling2D` میانگین هر کانال محاسبه می شود. این کار تعداد پارامترها را به شدت کاهش می دهد و از بیش برآزش جلوگیری می کند. سپس نیمی از نورون ها را در طول آموزش غیرفعال می کند.

سپس لایه آخر مدل را مشخص می کنیم. تمام لایه های میانی که بین این دو قرار گرفته اند به طور خودکار بخشی از مدل می شوند.

لایه خروجی با ۱۰ نورون و تابع `softmax` احتمال هر کلاس را تولید می کند. `Regularization` نیز یک جمله جریمه به تابع هزینه اضافه می کند که با مجذور اندازه وزن ها متناسب است. این جریمه مانع از آن می شود که مدل به هر ویژگی اهمیت بیش از حد بدهد و در نتیجه به جلوگیری از بیش برآزش کمک می کند.

```

165 x = residual_block(x, filters_base)
166 x = residual_block(x, filters_base)
167 x = residual_block(x, filters_base*2, stride=2)
168 x = residual_block(x, filters_base*2)
169 x = residual_block(x, filters_base*4, stride=2)
170 x = residual_block(x, filters_base*4)
171
172 x = layers.GlobalAveragePooling2D()(x) # same size as channel nums
173 x = layers.Dropout(dropout_rate)(x)
174 outputs = layers.Dense(num_classes, activation='softmax', kernel_regularizer=regularizers.l2(1e-4))(x) # avoid overfitting
175
176 model = models.Model(inputs, outputs)
177 return model

```

شکل ۱۱-۳، ۵/۱۴ پیاده‌سازی مدل cnn جدید با residual blocks

۳-۶. آموزش مدل.

حال دوباره بخش جستجوی هایپرپارامتر را اجرا می‌کنیم. این بار داده‌های آموزش افزایش‌یافته و معماری جدید مبتنی بر Residual Blocks مورد استفاده قرار گرفت. در این جستجو، هر ترکیب ۱۲ epoch آموزش داده می‌شود و patience مربوط به Early Stopping برابر با ۵ است.

بنابراین، ترکیب دوم به‌عنوان بهترین ترکیب انتخاب شد.

```

804/804 - 1248s - 2s/step - accuracy: 0.7832 - loss: 0.6665 - val_accuracy: 0.7461 - val_loss: 0.8472 - learning_rate: 1.2500e-04
Epoch 10/12
804/804 - 1249s - 2s/step - accuracy: 0.7949 - loss: 0.6355 - val_accuracy: 0.6923 - val_loss: 1.0921 - learning_rate: 1.2500e-04
Epoch 11/12
804/804 - 1252s - 2s/step - accuracy: 0.8021 - loss: 0.6079 - val_accuracy: 0.7341 - val_loss: 0.9338 - learning_rate: 1.2500e-04
Epoch 12/12
804/804 - 1251s - 2s/step - accuracy: 0.8202 - loss: 0.5555 - val_accuracy: 0.7493 - val_loss: 0.9139 - learning_rate: 6.2500e-05
best val_loss = 0.8472

search results
{'lr': 0.0005, 'batch_size': 16, 'dropout': 0.3, 'filters_base': 32} val_loss=0.6586best
{'lr': 0.001, 'batch_size': 16, 'dropout': 0.3, 'filters_base': 48} val_loss=0.6916
{'lr': 0.001, 'batch_size': 32, 'dropout': 0.5, 'filters_base': 32} val_loss=0.8252
{'lr': 0.0005, 'batch_size': 64, 'dropout': 0.5, 'filters_base': 32} val_loss=0.8472
{'lr': 0.01, 'batch_size': 64, 'dropout': 0.4, 'filters_base': 48} val_loss=0.8987

best hparams saved to best_hyperparams.txt

```

شکل ۱۲-۳، نتیجه جستجوی هایپرپارامترهای گسترده‌تر

سپس مدل را با ۱۵۰ epoch اجرا کردیم با هایپرپارامترهای مشخص شده ولی به‌خاطر وجود Early Stopping مدل در epoch شماره ۳۵ متوقف می‌شود.

```
Epoch 35: ReduceLROnPlateau reducing learning rate to 7.812500371073838e-06.
1340/1340 1039s 775ms/step - accuracy: 0.9332 - loss: 0.2167 - val_accuracy: 0.8025 - val_loss: 0.9851 - learning_rate: 1.5625e-05
Epoch 35: early stopping
Restoring model weights from the end of the best epoch: 23.
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Best model saved to best_model.h5
Evaluating on test set...
Found 1843 images belonging to 10 classes.
58/58 9s 161ms/step - accuracy: 0.7770 - loss: 1.0093
Test accuracy: 0.7770 | Test loss: 1.0093
```

شکل ۱۳-۳، خروجی مدل پس از epoch ۱۵۰ با early stopping

همانطور که مشخص است بر اساس کمترین خطای اعتبارسنجی، epoch شماره ۲۳ بهترین عملکرد را داشت با $val_loss = 0.8756$ و $val_accuracy = 0.8074$.

نتیجه اجرای مدل روی داده‌های تست:

دقت تست: ۰.۷۷۷۰

خطای تست: ۱.۰۰۹۳

در مقایسه با مدل اولیه دقت تست مدل افزایش یافته و خطا کاهش یافته است اما نسبت به مدل دوم، دقت تست کمتری شده.

جدول ۳-۳. نتایج اجرای مدل ۱، ۲ و ۳

detail		Test loss	Test acc	Val loss	Val acc	ردیف
cnn	hypram					
40 ep	Lr, batch size	۱.۰۸۱۵	۰.۶۵۱۱	۱.۰۵۰۶	۰.۶۵۸۲	۱
120 ep + early stopping	Lr, batch size, dropout, filter base	۰.۹۵۹۵	۰.۸۳۹۴	۰.۸۲۴۹	۰.۷۹۸۷	۲
Res + ۱۵۰ ep + early stopping	Lr, batch size, dropout, filter base	۱.۰۰۹۳	۰.۷۷۷۰	۰.۸۷۵۶	۰.۸۰۷۴	۳

با وجود استفاده از Residual Blocks و افزایش داده، نتوانستیم دقت تست بهتری نسبت به مدل دوم به دست آوریم پس ممکن است Early Stopping بیش از حد زود فعال شده باشد. به همین دلیل، مدل را بدون Early Stopping دوباره آموزش می‌دهیم.

مدل را با epoch ۱۵۰ اجرا کردیم با هایپرپارامترهای مشخص شده بدون Early

.Stopping

```
Epoch 150: val_loss did not improve from 0.92633
1340/1340 150s/step - accuracy: 0.9570 - loss: 0.1469 - val_accuracy: 0.8095 - val_loss: 1.0099 - learning_rate: 1.0000e-07
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We
e recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Best model saved to best_model.h5

Evaluating on test set...
Found 1843 images belonging to 10 classes.
58/58 10s 180ms/step - accuracy: 0.8106 - loss: 1.0768
Test accuracy: 0.8106 | Test loss: 1.0768
```

شکل ۱۴-۳، خروجی مدل پس از ۱۵۰ epoch بدون early stopping

بر اساس کمترین خطای اعتبارسنجی، epoch شماره ۸ بهترین عملکرد را داشت با $val_accuracy = 0.8627$ و $val_loss = 0.7444$.

نتیجه اجرای مدل روی داده‌های تست:

دقت تست: ۰.۸۱۰۶

خطای تست: ۱.۰۷۶۸

جدول ۴-۳. نتایج اجرای مدل ۱، ۲، ۳ و ۴

detail		Test loss	Test acc	Val loss	Val acc	ردیف
cnn	hypram					
40 ep	Lr, batch size	۱.۰۸۱۵	۰.۶۵۱۱	۱.۰۵۰۶	۰.۶۵۸۲	۱
120 ep + early stopping	Lr, batch size, dropout, filter base	۰.۹۵۹۵	۰.۸۳۹۴	۰.۸۲۴۹	۰.۷۹۸۷	۲
Res + ۱۵۰ ep + early stopping	Lr, batch size, dropout, filter base	۱.۰۰۹۳	۰.۷۷۷۰	۰.۸۷۵۶	۰.۸۰۷۴	۳
Res + ۱۵۰ ep	Lr, batch size, dropout, filter base	۱.۰۷۶۸	۰.۸۱۰۶	۰.۷۴۴۴	۰.۸۶۲۷	۴

با توجه به نتایج جدول، در مدل چهارم دقت اعتبارسنجی نسبت به اجرای سوم بهبود داشت. این بهبود نشان می‌دهد که مدل با ادامه‌ی آموزش و کاهش تدریجی نرخ یادگیری توانسته است به نواحی بهتری از فضای وزن‌ها همگرا شود. اما دقت تست همچنان از مدل دوم پایین‌تر است پس به همین دلیل منطقی است که فرصت بیشتری به مدل بدهیم.

در مدل‌های ۳ و ۴ داده‌های اصلی به پوشه داده‌های تغییر یافته اضافه نشده بود حال خود تصویر اصلی نیز به همان پوشه‌ی مقصد کپی می‌شود تا در کنار نسخه‌های تغییر یافته، همه‌ی تصاویر برای آموزش در دسترس باشند.

سپس دوباره بخش جستجوی هایپرپارامتر را اجرا کردیم. این بار داده‌های آموزش افزایش یافته و داده‌های اصلی.

در این جستجو، هر ترکیب ۱۲ epoch آموزش داده می‌شود و patience مربوط به Early Stopping برابر با ۵ است و ترکیب اول به‌عنوان بهترین ترکیب انتخاب شد.

مدل را با epoch ۲۵۰ اجرا کردیم با هایپرپارامترهای مشخص شده بدون Early Stopping.

```
Epoch 249: val_loss did not improve from 0.72879
3214/3214 2182s 679ms/step - accuracy: 0.9787 - loss: 0.0813 - val_accuracy: 0.8660 - val_loss: 0.7590 - learning_rate: 1.0000e-07
Epoch 250/250
3214/3214 0s 676ms/step - accuracy: 0.9803 - loss: 0.0773
Epoch 250: saving model to checkpoints\model_epoch_250.h5
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
Epoch 250: val_loss did not improve from 0.72879
3214/3214 2198s 684ms/step - accuracy: 0.9790 - loss: 0.0801 - val_accuracy: 0.8660 - val_loss: 0.7442 - learning_rate: 1.0000e-07
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save('my_model.keras')' or 'keras.saving.save_model(model, 'my_model.keras')'.
best model saved to best_model.h5

Evaluating on test set
Found 1843 images belonging to 10 classes.
58/58 21s 367ms/step - accuracy: 0.8486 - loss: 0.8924
test accuracy: 0.8486, test loss: 0.8924
```

شکل ۱۵-۳، خروجی مدل پس از epoch ۲۵۰ بدون early stopping

بر اساس کمترین خطای اعتبارسنجی، epoch شماره ۲۶ بهترین عملکرد را داشت با $val_accuracy = 0.8535$ و $val_loss = 0.6786$.

نتیجه اجرای مدل روی داده‌های تست:

دقت تست: ۰.۸۴۸۶

خطای تست: ۰.۸۹۲۴

جدول ۳-۵. نتایج اجرای مدل ۱، ۲، ۳، ۴ و ۵

detail		Test loss	Test acc	Val loss	Val acc	ردیف
cnn	hypram					

40 ep	Lr, batch size	۱.۰۸۱۵	۰.۶۵۱۱	۱.۰۵۰۶	۰.۶۵۸۲	۱
120 ep + early stopping	Lr, batch size, dropout, filter base	۰.۹۵۹۵	۰.۸۳۹۴	۰.۸۲۴۹	۰.۷۹۸۷	۲
Res + ۱۵۰ ep + early stopping	Lr, batch size, dropout, filter base	۱.۰۰۹۳	۰.۷۷۷۰	۰.۸۷۵۶	۰.۸۰۷۴	۳
Res + ۱۵۰ ep	Lr, batch size, dropout, filter base	۱.۰۷۶۸	۰.۸۱۰۶	۰.۷۴۴۴	۰.۸۶۲۷	۴
Res + ۲۵۰ ep	Lr, batch size, dropout, filter base	۰.۸۹۲۴	۰.۸۴۸۶	۰.۶۷۸۶	۰.۸۵۳۵	۵

جدول روند بهبود تدریجی مدل را نشان می‌دهد که با گسترش جستجوی هایپرپارامترها و افزایش epoch دقت تست و خطای تست پیشرفت کرد. سپس با تغییر معماری به Residual Blocks و افزایش epoch نتایج ابتدا دچار افت شدند و در نهایت، با حذف Early Stopping و افزایش epoch، بهترین نتیجه به‌دست آمد.

اما نکته‌ی مهم این است که در مدل ۴، خطای اعتبارسنجی کاهش یافت، اما دقت تست همچنان از مدل ۲ کمتر بود. این نشان می‌دهد که معیار اعتبارسنجی همیشه بهترین نماینده‌ی عملکرد واقعی روی داده‌های تست نیست و ممکن است مدل روی اعتبارسنجی بیش‌برازش کند.

۴. معیار FoM

۴-۱. تعریف FoM

یکی از وظایف کلیدی در یادگیری ماشین، بهینه‌سازی مدل‌ها در مراحل مختلف است؛ برای نمونه انتخاب هایپرپارامترها یا تعیین نقطه توقف آموزش. رویکرد سنتی در این زمینه، استفاده از خطای اعتبارسنجی است. با این حال، یادگیری ماشین در حوزه سلامت هدفی متفاوت از یادگیری ماشین سنتی دارد: مدل باید بر اساس نیازهای مشخص و کاربردی عملکرد خوبی داشته باشد، نه صرفاً بر اساس تابع خطایی که برای آموزش استفاده شده است. این نیازها را می‌توان با معیارهای سفارشی‌سازی‌شده دقیق‌تر بیان کرد [18].

در طبقه‌بندی تصاویر زبلله نیز همین منطق برقرار است؛ برخی کلاس‌ها مانند باتری اهمیت بیشتری نسبت به کلاس‌های دیگر دارند. دقت کلی مدل ممکن است بالا باشد، اما اگر مدل در تشخیص این کلاس‌های حساس ضعیف عمل کند، سیستم برای کاربرد واقعی مناسب نیست. بنابراین معیار (FoM) figure of merit را تعریف کردیم تا عملکرد مدل را روی همین کلاس‌های مهم بسنجد.

۴-۲. پیاده‌سازی FoM

پس تابعی به نام `task_fom` تعریف کردیم که میانگین `recall` را برای سه کلاس خطرناک محاسبه می‌کند. `recall` نشان می‌دهد که از بین نمونه‌های واقعی مربوط به هر کلاس خطرناک، چه تعدادی به درستی توسط مدل شناسایی شده‌اند.

برای محاسبه مقدار `recall` برای هر کلاس، از فرمول زیر استفاده می‌کنیم:

$$Recall = \frac{TP}{TP+FN} \quad 4$$

که در آن TP تعداد نمونه‌هایی است که مدل به‌درستی تشخیص داده (مثبت واقعی) و FN تعداد نمونه‌هایی است که مدل به‌اشتباه تشخیص نداده (منفی کاذب) می‌باشد.

بالا بودن این معیار یعنی مدل تعداد بیشتری از زباله‌های خطرناک را درست تشخیص می‌دهد و موارد کمتری را از دست می‌دهد. میانگین‌گیری از recall سه کلاس نیز باعث می‌شود مدل فقط روی یک کلاس خوب عمل نکند و عملکرد متوازنی روی همه‌ی کلاس‌های حساس داشته باشد.

```
158 def task_fom(y_true, y_pred):
159     recalls = recall_score(y_true, y_pred, labels=hazard_classes, average=None) # recall for hazard_classes only
160     return np.mean(recalls) # higher better
161
```

شکل ۱-۴، پیاده‌سازی تابع task_fom

برای این‌که این معیار در طول آموزش محاسبه شود، یک callback جدید ساختیم. این callback پس از پایان هر epoch، پیش‌بینی مدل را روی کل مجموعه‌ی اعتبارسنجی انجام می‌دهد، برچسب‌های واقعی را با پیش‌بینی‌ها مقایسه می‌کند و مقدار val_fom را محاسبه می‌کند.

همه Callback های سفارشی از کلاس keras.callbacks.Callback ارث‌بری می‌کنند و متدهایی مانند on_epoch_end و on_train_end را بازنویسی می‌کنند. سپس Callback به model.fit اضافه می‌شود. در متدهایی مانند on_epoch_end یا on_train_batch_end، پارامتر logs شامل مقادیر معیارها است.

Callback ها از طریق self.model به مدل دسترسی دارند. با استفاده از آن می‌توان آموزش را متوقف کرد، بهینه‌ساز را تغییر داد، مدل را ذخیره کرد یا پیش‌بینی انجام داد [19].

```

162 class CustomMetricCallback(tf.keras.callbacks.Callback):
163     def __init__(self, val_gen): # based on val
164         super().__init__()
165         self.val_gen = val_gen
166
167     def on_epoch_end(self, epoch, logs=None):
168         self.val_gen.reset()
169         y_true = self.val_gen.classes
170         y_pred = np.argmax(self.model.predict(self.val_gen, verbose=0), axis=1)
171         fom = task_fom(y_true, y_pred)
172         logs['val_fom'] = fom
173         print(f"val_fom: {fom:.4f}")
174

```

شکل ۲-۴، پیاده‌سازی callback سفارشی

معیار نظارت در callbackها از val_loss به val_fom تغییر یافته است. ModelCheckpoint به جای val_loss روی val_fom نظارت می‌کند و چون هدف پیشینه‌کردن FoM است، از حالت mode=max استفاده می‌شود. EarlyStopping نیز بر اساس val_fom و با mode=max تنظیم شده است به این معنا که اگر FoM برای تعداد مشخصی epoch بهبود نیابد، فرایند آموزش متوقف شده و بهترین وزن‌های ذخیره‌شده بازیابی می‌شوند. ReduceLRonPlateau همچنان روی val_loss عمل می‌کند تا نرخ یادگیری را در صورت عدم بهبود خطای اعتبارسنجی کاهش دهد.

```

callbacks = [
    CustomMetricCallback(val_gen),
    tf.keras.callbacks.ModelCheckpoint(filepath=os.path.join(ckpt_dir, 'model_epoch_{epoch:02d}.h5'), save_weights_only=False, save_freq='epoch', verbose=1),
    tf.keras.callbacks.ModelCheckpoint(filepath=best_model_path, monitor='val_fom', mode='max', save_best_only=True, verbose=1),
    tf.keras.callbacks.EarlyStopping(monitor='val_fom', patience=early_stopping_p, mode='max', restore_best_weights=True, verbose=1),
    tf.keras.callbacks.ReduceLRonPlateau(monitor='val_loss', factor=lr_value, patience=lr_p, min_lr=min_lr, verbose=1)
]

```

شکل ۳-۴، تغییر callbackها برای کار با fom

سه کلاس زیر به عنوان کلاس‌های خطرناک در نظر گرفته شده‌اند:

- Battery: زیرا حاوی مواد شیمیایی است و اگر به درستی تفکیک و بازیافت نشوند، می‌توانند باعث آلودگی شدید خاک و آب شوند.
- Biological: زیرا شامل باقی‌مانده غذا، پوست میوه و گیاهان است و اگر این مواد با سایر زباله‌ها مخلوط شوند، می‌توانند باعث جذب حشرات و گسترش بیماری شوند.
- Trash: این کلاس به زباله‌هایی اشاره دارد که قابل بازیافت نیستند.

برای تقویت یادگیری همین کلاس‌های خطرناک، وزن آن‌ها در تابع هزینه سه برابر سایر

کلاس‌ها در نظر گرفته شده است. این کار باعث می‌شود خطای مربوط به این کلاس‌ها تأثیر بیشتری در به‌روزرسانی وزن‌ها داشته باشد و مدل توجه بیشتری به تشخیص صحیح آن‌ها نشان دهد.

تابع `evaluate_test` مدل آموزش‌دیده را به‌عنوان ورودی دریافت می‌کند و عملکرد آن را روی مجموعه تست ارزیابی می‌کند. مدل روی تمام تصاویر تست اجرا می‌شود و مقادیر `loss` و `accuracy` محاسبه می‌شوند.

برای بررسی FoM روی داده‌های تست، ابتدا برچسب‌های واقعی تصاویر تست استخراج می‌شوند و سپس خروجی مدل برای هر تصویر تست به دست می‌آید. در ادامه، با فراخوانی تابع `task_fom` میانگین `recall` سه کلاس خطرناک محاسبه می‌شود. این مقدار نشان می‌دهد مدل چقدر توانسته زباله‌های خطرناک را روی داده‌های تست شناسایی کند.

```

339 def evaluate_test(model):
340     test_datagen = ImageDataGenerator(rescale=1./255)
341     test_gen = test_datagen.flow_from_directory(os.path.join(split_ds, 'test'), target_size=(img_h, img_w), batch_size=32, class_mode='categorical', shuffle=False)
342     loss, acc = model.evaluate(test_gen, verbose=1)
343     print(f"test accuracy: {acc:.4f}, test loss: {loss:.4f}")
344
345     test_gen.reset() # fom
346     y_true = test_gen.classes
347     y_pred = np.argmax(model.predict(test_gen, verbose=0), axis=1)
348     fom = task_fom(y_true, y_pred)
349     print(f"test task FOM (hazard recall): {fom:.4f}")
350     return loss, acc

```

شکل ۴-۴، پیاده‌سازی تابع `evaluate_test`

۳-۴. آموزش مدل.

دوباره بخش جستجوی هایپرپارامتر را اجرا کردیم. این بار با معیار FoM.

در این جستجو، هر ترکیب ۱۲ epoch آموزش داده می‌شود و `patience` مربوط به `Early Stopping` برابر با ۵ است و ترکیب دوم به‌عنوان بهترین ترکیب انتخاب شد.

مدل را با ۲۰۰ epoch اجرا کردیم با هایپرپارامترهای مشخص شده بدون `EarlyStopping`.

```

evaluating on test set
Found 1863 Images belonging to 10 classes.
C:\Users\test\AppData\Roaming\Python\Python313\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your 'PyDataset' class should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not pass these arguments to 'fit()', as they will be ignored.
  self._warn_if_super_not_called()
58/58 15s 261ms/step - accuracy: 0.8199 - loss: 0.8196
test accuracy: 0.8199, test loss: 0.8196
test task FOM (hazard recall): 0.7265

```

شکل ۴-۵، خروجی مدل پس از ۲۰۰ epoch بدون `early stopping`

بر اساس بیشترین FoM، epoch شماره ۱۴، بهترین عملکرد را داشت با $val_loss = 0.7491$ و $val_accuracy = 0.8020$ و $val_fom = 0.8040$.

نتیجه اجرای مدل روی داده‌های تست:

دقت تست: ۰.۸۱۰۱

خطای تست: ۰.۸۹۵۱

FoM تست: ۰.۸۱۱۹

جدول ۴-۱. نتایج اجرای مدل ۱، ۲، ۳، ۴، ۵ و ۶

ردیف	Val acc	Val loss	Test acc	Test loss	Val FoM	Test FoM	detail
							cnn
۱	۰.۶۵۸۲	۱.۰۵۰۶	۰.۶۵۱۱	۱.۰۸۱۵	-	-	40 ep
۲	۰.۷۹۸۷	۰.۸۲۴۹	۰.۸۳۹۴	۰.۹۵۹۵	-	-	120 ep + early stopping
۳	۰.۸۰۷۴	۰.۸۷۵۶	۰.۷۷۷۰	۱.۰۰۹۳	-	-	Res + ۱۵۰ ep + early stopping
۴	۰.۸۶۲۷	۰.۷۴۴۴	۰.۸۱۰۶	۱.۰۷۶۸	-	-	Res + ۱۵۰ ep
۵	۰.۸۵۳۵	۰.۶۷۸۶	۰.۸۴۸۶	۰.۸۹۲۴	-	-	Res + ۲۵۰ ep
۶	۰.۸۰۲۰	۰.۷۴۹۱	۰.۸۱۰۱	۰.۸۹۵۱	۰.۸۰۴۰	۰.۸۱۹۹	FoM + Res + ۲۰۰ ep

دقت تست نسبت به مدل ۵ کمی کاهش پیدا کرد، اما روند نتایج نشان می‌دهد که افزایش تعداد epoch تا این‌جا تأثیر مثبتی بر عملکرد مدل داشته است. پس همین مدل را یکبار دیگر با تعداد epoch بیشتری اجرا کنیم.

مدل را با ۳۰۰ epoch اجرا کردیم با هایپرپارامترهای مشخص شده بدون Early Stopping.

```

evaluating on test set
Found 1843 images belonging to 10 classes.
C:\Users\est\AppData\Roaming\Python\Python313\site-packages\keras\src\trainers\data_adapters\py_dataset_adapter.py:121: UserWarning: Your 'PyDataset'
lass should call 'super().__init__(**kwargs)' in its constructor. '**kwargs' can include 'workers', 'use_multiprocessing', 'max_queue_size'. Do not p
s these arguments to 'fit()', as they will be ignored.
  self._warn_if_super_not_called()
58/58 — 45s 775ms/step - accuracy: 0.8101 - loss: 0.8951
test accuracy: 0.8101, test loss: 0.8951
test task FoM (hazard recall): 0.8119

```

شکل ۶-۴، خروجی مدل پس از ۳۰۰ epoch بدون early stopping

بر اساس بیشترین FoM، epoch شماره ۱۹، بهترین عملکرد را داشت با
 $val_fom = 0.8659$ و $val_accuracy = 0.8253$ و $val_loss = 0.7623$

نتیجه اجرای مدل روی داده‌های تست: دقت تست: ۰.۸۰۸۵ و خطای تست: ۰.۷۹۱۷.

جدول ۲-۴. نتایج اجرای تمام مدل‌ها

detail	Test FoM	Val FoM	Test loss	Test acc	Val loss	Val acc	ردیف
cnn							
40 ep	-	-	۱.۰۸۱۵	۰.۶۵۱۱	۱.۰۵۰۶	۰.۶۵۸۲	۱
120 ep + early stopping	-	-	۰.۹۵۹۵	۰.۸۳۹۴	۰.۸۲۴۹	۰.۷۹۸۷	۲
Res + ۱۵۰ ep + early stopping	-	-	۱.۰۰۹۳	۰.۷۷۷۰	۰.۸۷۵۶	۰.۸۰۷۴	۳
Res + ۱۵۰ ep	-	-	۱.۰۷۶۸	۰.۸۱۰۶	۰.۷۴۴۴	۰.۸۶۲۷	۴
Res + ۲۵۰ ep	-	-	۰.۸۹۲۴	۰.۸۴۸۶	۰.۶۷۸۶	۰.۸۵۳۵	۵
FoM + Res + ۲۰۰ ep	۰.۸۱۹۹	۰.۸۰۴۰	۰.۸۹۵۱	۰.۸۱۰۱	۰.۷۴۹۱	۰.۸۰۲۰	۶
FoM + Res + ۳۰۰ ep	۰.۸۴۲۳	۰.۸۶۵۹	۰.۷۹۱۷	۰.۸۰۸۵	۰.۷۶۲۳	۰.۸۲۵۳	۷

مدل ۷ مقدار FoM بهتری نسبت به مدل ۶ دارد و خطای تست آن در مقایسه با بقیه‌ی مدل‌ها کمتر است. دقت تست این مدل از مدل ۵ کمتر است اما چون هدف تشخیص درست زباله‌های حساس و خطرناک است، معیار FoM اهمیت بیشتری از دقت کلی دارد. به همین دلیل، مدل ۷ به‌عنوان مدل نهایی انتخاب می‌شود.

۵. تشخیص چند object در تصویر.

۵-۱. بیان مسئله.

در کاربرد واقعی تفکیک زباله، تصاویر معمولاً شامل چندین زباله با انواع مختلف هستند بنابراین لازم است مدل بتواند چند object را در یک تصویر شناسایی کند و هر کدام را جداگانه طبقه‌بندی نماید تا در سیستم‌های واقعی تفکیک زباله، هر object به‌درستی تشخیص داده شده و به مسیر مناسب هدایت شود.

چون مدل با داده‌های multiple objects آموزش داده نشده است، پس نمی‌توان به آن تصاویر با multiple objects داد و انتظار داشت که پاسخ درستی بدهد. برای انجام این کار، باید ورودی‌ای که شامل multiple objects است را به بخش‌هایی تقسیم کنیم و هر بخش را بررسی کنیم که آیا یک object خاص در آن وجود دارد یا نه. برای این کار، پنجره‌هایی روی کل تصویر حرکت می‌کنند که مشخصاً تعداد این پنجره‌ها زیاد می‌شود پس از متد selective search OpenCV استفاده می‌کنیم. این روش لیستی از مستطیل‌ها را برمی‌گرداند که در واقع همان نواحی مورد نظر (Region of Interest) هستند [20].

۵-۲. پیاده‌سازی.

تابع زیر تصویر ورودی را با Selective Search پردازش می‌کند و لیستی از مستطیل‌ها را به‌عنوان نواحی پیشنهادی برمی‌گرداند.

```

386 def generate_regions(img_bgr, max_proposals=max_reg):
387     search = cv2.ximgproc.segmentation.createSelectiveSearchSegmentation()
388     search.setBaseImage(img_bgr) # set the base image as the input image
389     search.switchToSelectiveSearchQuality()
390     rects = search.process() # process the image
391     return rects[:max_proposals]

```

شکل ۵-۱، پیاده‌سازی تابع generate_regions

تابع زیر نواحی تولیدشده توسط Selective Search را دریافت می‌کند و نواحی بسیار کوچک را حذف می‌کند. سپس هر ناحیه از تصویر اصلی برش داده شده و برای ورود به مدل آماده می‌شود. خروجی شامل تصاویر پردازش‌شده و مختصات هر ناحیه است.

```

393 def create_box(img_bgr, rects, img_w=224, img_h=224, min_ratio=0.1):
394     rois = []
395     boxes = []
396     (H, W) = img_bgr.shape[:2]
397
398     for (x, y, w, h) in rects:
399         if w / float(W) < min_ratio or h / float(H) < min_ratio: # if roi smaller than 10% of img get rid of it
400             continue
401
402         roi = img_bgr[y:y+h, x:x+w] # extract the Roi from image
403         if roi.size == 0:
404             continue
405         roi = cv2.cvtColor(roi, cv2.COLOR_BGR2RGB) # to rgb
406         roi = cv2.resize(roi, (img_w, img_h))
407         roi = roi.astype('float32') / 255.0
408
409         rois.append(roi)
410         x1, y1, x2, y2 = x, y, x+w, y+h # box coordinates
411         boxes.append((x1, y1, x2, y2))
412
413     return rois, boxes

```

شکل ۵-۲، پیاده‌سازی تابع create_box

حال باید تصاویر تولیدشده را به مدل بدهیم تا پیش‌بینی کند. تابع زیر این کار را انجام می‌دهد.

```

415 def predict_rios(model, rois):
416     rois_array = np.array(rois) # rois to np array for model (w, h, c, number)
417     preds = model.predict(rois_array, verbose=0) # predict all rois the same time
418     class_indices = np.argmax(preds, axis=1) # best indx for each rio
419     confidences = np.max(preds, axis=1) # best value for each rio
420
421     return class_indices, confidences

```

شکل ۵-۳، پیاده‌سازی تابع predic_rios

سپس با استفاده از تابع زیر یک دیکشنری جدید ساخته می‌شود که key آن نام کلاس و

value آن لیستی از box و conf مربوط به همان کلاس است. با این کار می‌توانیم پیش‌بینی‌های هر کلاس را به راحتی در دسترس داشته باشیم و non_max_suppression را به صورت جداگانه روی هر کلاس اعمال کنیم. در این فرایند، فقط مواردی که اطمینان آن‌ها از آستانه‌ی مشخص شده بیشتر است نگه داشته می‌شوند تا نویز و تشخیص‌های ضعیف حذف شوند.

```

423 def build_objects(boxes, class_indices, confidences, confidence_thresh=confidence_thresh):
424     objects = {}
425     for box, cls_idx, conf in zip(boxes, class_indices, confidences):
426         if conf < confidence_thresh:
427             continue
428         label = class_name[cls_idx]
429         objects.setdefault(label, []).append((box, conf))
430     return objects

```

شکل ۴-۵، پیاده‌سازی تابع build_objects

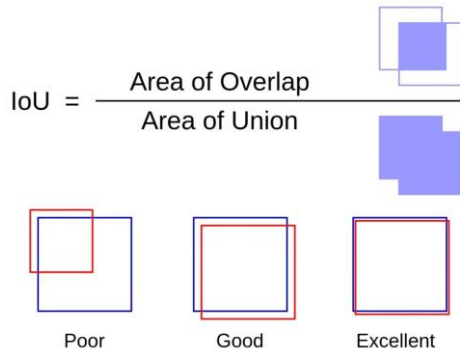
در الگوریتم‌های object detection به‌ویژه روش‌هایی که از Sliding Window استفاده می‌کنند، برای یک object چندین Bounding Box تولید می‌شود. این تشخیص‌های هم‌پوشان، خروجی نهایی را شلوغ کرده و تشخیص را دشوار می‌سازند.

Non-Maximum Suppression یک مرحله‌ی پس‌پردازش در object detection است که تضمین می‌کند هر object تشخیص داده شده تنها با یک Bounding Box نمایش داده شود.

معیار IoU یا Intersection over Union، میزان هم‌پوشانی بین دو Bounding Box را اندازه‌گیری می‌کند. فرمول آن به صورت زیر است:

$$IoU = \frac{A \cap B}{A \cup B} \quad 5$$

که در آن A و B نشان‌دهنده‌ی دو Bounding Box هستند. مقدار IoU همیشه بین ۰ و ۱ قرار دارد و هرچه به ۱ نزدیک‌تر باشد، هم‌پوشانی بیشتر است.



شکل ۵-۵، IoU [21]

تابع زیر برای حذف جعبه‌های overlap به کار می‌رود. این تابع دو لیست را دریافت می‌کند boxes که شامل مختصات هر جعبه و scores که شامل امتیاز متناظر با هر جعبه است. اگر هم‌پوشانی دو جعبه از nms_thresh بیشتر باشد، جعبه با امتیاز پایین‌تر حذف می‌شود.

```

432 def apply_nms(boxes, scores, score_thresh=0.0, nms_thresh=iou_thresh): # score_thresh=0.0 cause its done in build_objects
433     cv2_boxes = []
434     # nms needs x,y,w,h
435     for (x1, y1, x2, y2) in boxes:
436         w = x2 - x1
437         h = y2 - y1
438         cv2_boxes.append([int(x1), int(y1), int(w), int(h)])
439
440     indices = cv2.dnn.NMSBoxes(cv2_boxes, scores, score_thresh, nms_thresh) # output= box, score
441
442     selected_indices = []
443     if len(indices) > 0:
444         for i in indices:
445             idx = i[0] if isinstance(i, (list, tuple)) else i
446             selected_indices.append(idx)
447
448     return selected_indices

```

شکل ۵-۶، پیاده‌سازی تابع apply_nms

تابع زیر دیکشنری objects را می‌گیرد که در آن جعبه‌ها و امتیازها بر اساس نام کلاس گروه‌بندی شده‌اند. سپس برای هر کلاس به صورت جداگانه nms را صدا می‌زنیم. این کار باعث می‌شود اگر چند Bounding Box مربوط به یک کلاس داشته باشیم، فقط بهترین آن‌ها باقی بماند و اگر دو object از کلاس‌های مختلف نزدیک هم باشند، چون در گروه‌های جداگانه بررسی می‌شوند، هیچ کدام به اشتباه حذف نمی‌شوند.

```

451 def apply_nms_per_class(objects, iou_thresh=iou_thresh): # based on class name
452     final_detections = []
453
454     for label, dets in objects.items():
455         boxes = [d[0] for d in dets] # list of (x1, y1, x2, y2)
456         scores = [d[1] for d in dets] # confidence scores
457
458         selected = apply_nms(boxes, scores, score_thresh=0.0, nms_thresh=iou_thresh)
459
460         for idx in selected:
461             x1, y1, x2, y2 = boxes[idx]
462             conf = scores[idx]
463             final_detections.append((x1, y1, x2, y2, conf, label))
464
465     return final_detections

```

شکل ۷-۵، پیاده‌سازی تابع `apply_nms_per_class`

تابع زیر تمامی مراحل برای multi object detection را اجرا می‌کند و در آخر bounding boxes را روی تصویر مشخص می‌کند.

```

467 def multi_objects_detection(model, img_path, confidence_thresh=confidence_thresh, iou_thresh=iou_thresh, max_proposals=max_reg):
468     img_bgr = cv2.imread(img_path)
469     if img_bgr is None:
470         raise FileNotFoundError(f"Image not found: {img_path}")
471
472     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB) # rgb for model
473
474     proposals = generate_regions(img_bgr, max_proposals)
475     rois, boxes = create_box(img_bgr, proposals)
476
477     if not rois:
478         print("no valid regions found.")
479         return img_rgb, []
480
481     class_ids, confs = predict_rois(model, rois)
482     objects = build_objects(boxes, class_ids, confs, confidence_thresh)
483     final_detections = apply_nms_per_class(objects, iou_thresh)
484
485     # draw boxes
486     for (x1, y1, x2, y2, conf, label) in final_detections:
487         color = (0, 255, 0) if label in usable else (255, 0, 0)
488         cv2.rectangle(img_rgb, (int(x1), int(y1)), (int(x2), int(y2)), color, 2) # (x1,y1) top left ,(x2,y2) bottom right
489         cv2.putText(img_rgb, f"{label} ({conf:.2f})", (int(x1), int(y1) - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2) # on top of the box
490
491     return img_rgb, final_detections

```

شکل ۸-۵، پیاده‌سازی تابع `multi_objects_detection`

۶. جمع‌بندی.

۶-۱. بررسی خروجی مدل روی ورودی با یک object.

تصویر زیر به مدل داده شد.



شکل ۱-۶، single object image

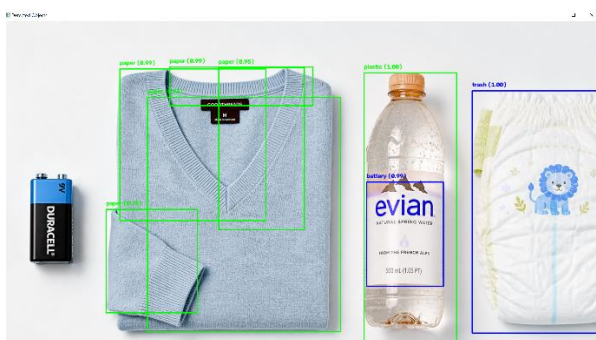
خروجی نشان می‌دهد که مدل در طبقه‌بندی تصاویری که تنها شامل یک object هستند عملکرد قابل‌قبولی دارد و می‌تواند به خوبی عمل کند.

```
test accuracy: 0.8423, test loss: 0.7521
test task FoM (hazard recall): 0.8423
image: split_dataset/test/trash/trash_70.jpg
prediction: not usable, trash, confidence: 0.9540
```

شکل ۲-۶، خروجی مدل با single object image

۶-۲. بررسی خروجی مدل روی ورودی با multi object.

تصویر multi object به مدل داده شد.



شکل ۳-۶، خروجی مدل با multi object image

همانطور که مشخص است تعداد زیادی box هم پوشان روی تصویر ایجاد شد که نتیجه را شلوغ و غیردقیق کرد و باتری به طور کامل از تشخیص جا ماند اما بعضی هم درست تشخیص داده شدند.

Non-Maximum Suppression به صورت جداگانه برای هر کلاس اعمال می شود بنابراین اگر دو ناحیه ی تقریباً یکسان به دو کلاس متفاوت نسبت داده شوند، هر دو باقی می ماند و روی هم می افتند و همچنین برای box های هم کلاس، آستانه ی IoU برابر با ۰.۴ در نظر گرفته شده است؛ اما الگوریتم Selective Search به طور ذاتی نواحی هم پوشان بسیار زیادی تولید می کند. در این حالت، IoU ممکن است بسیار کمتر از ۰.۴ باشد و در نتیجه هر دو جعبه حفظ می شوند.

Selective Search نیز همیشه برای object های کوچک، کم کنتراست یا چسبیده به پس زمینه، ناحیه ی پیشنهادی مناسبی تولید نمی کند. در نتیجه مدل هرگز فرصت طبقه بندی آن بخش را پیدا نمی کند.

۳-۶. نتیجه.

بهتر بود مدل از ابتدا برای تشخیص چند object آموزش داده می شد که مستقیماً bounding box ها را تولید کند و به صورت طبیعی با مشکل هم پوشانی و نادیده گرفتن object بهتر کنار بیاید. در کنار این، می توان آستانه ی IoU را به کاهش داد و آستانه ی اطمینان را بالاتر افزایش داد. افزودن یک مرحله ی NMS پس از NMS کلاس به کلاس و

حذف جعبه‌های تو در تو بر اساس معیار مرکز نیز به کاهش هم‌پوشانی‌های اضافی کمک می‌کند. به‌کارگیری این راهکارها می‌تواند خروجی تشخیص را تمیزتر و کامل‌تر کند و سیستم را برای استفاده در محیط‌های واقعی آماده‌تر کند.

٧. فهرست منابع

- [1] "Garbage Dataset," [Online]. Available: <https://www.kaggle.com/datasets/sumn2u/garbage-classification-v2>. [Accessed 15 August 2026].
- [2] "Kaggle: The World's AI Proving Ground," [Online]. Available: <https://www.kaggle.com/>. [Accessed 15 August 2026].
- [3] R. Pramoditha, "The Concept of Artificial Neurons (Perceptrons) in Neural Networks," Towards Data Science, [Online]. Available: <https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/>. [Accessed 15 August 2026].
- [4] A. Thevenot, "Conv2d: Finally Understand What Happens in the Forward Pass," Medium, [Online]. Available: <https://medium.com/data-science/conv2d-to-finally-understand-what-happens-in-the-forward-pass-1bbaafb0b148>. [Accessed 15 August 2026].
- [5] Crypto1, "Gradient Descent Algorithm: How Does it Work in Machine Learning?," Analytics Vidhya, [Online]. Available: <https://www.analyticsvidhya.com/blog/2020/10/how-does-the-gradient-descent-algorithm-work-in-machine-learning/>. [Accessed 15 August 2026].
- [6] "A Gentle Introduction to Learning Rate Schedulers," MachineLearningMastery.com, [Online]. Available: <https://machinelearningmastery.com/a-gentle-introduction-to-learning-rate-schedulers/>. [Accessed 15 August 2026].
- [7] "ReLU Activation Function in Deep Learning," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/relu-activation-function-in-deep-learning/>. [Accessed 15 August 2026].
- [8] "Guide to the Rectified Linear Unit (ReLU)," DataCamp, [Online]. Available: <https://www.datacamp.com/blog/rectified-linear-unit-relu>. [Accessed 15 August 2026].
- [9] N. Nguyen, N. Nguyen, D. Ngo, T. Ngo and L. Pham, "An Analysis of State-of-the-art Activation Functions For Supervised Deep Neural Network," *arXiv*, 2021.
- [10] "Apply a 2D Max Pooling in PyTorch," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/computer-vision/apply-a-2d-max-pooling-in-pytorch/>. [Accessed 15 August 2026].
- [11] D. Landup, "Don't Use Flatten() - Global Pooling for CNNs with TensorFlow and Keras," stackabuse.com, [Online]. Available: <https://stackabuse.com/dont-use-flatten-global-pooling-for-cnns-with-tensorflow-and-keras/>. [Accessed 15 August 2026].
- [12] "Introduction to Pooling Layer in CNN," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/introduction-to-pooling-layer-cnn/>. [Accessed 15 August 2026].
- [13] "What is Fully Connected Layer in Deep Learning," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/what-is-fully-connected-layer-in-deep-learning/>. [Accessed 15 August 2016].
- [14] K. Team, "Keras documentation: ReduceLROnPlateau," [Online]. Available: https://keras.io/api/callbacks/reduce_lr_on_plateau/. [Accessed 15 August 2026].
- [15] "model.fit() in TensorFlow," GeeksforGeeks, [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/model-fit-in-tensorflow/>. [Accessed 15 August 2026].

2026].

- [16] X. Liu and K. M. Goh, "ResNet: Enabling Deep Convolutional Neural Networks through Residual Learning," *arXiv*, 2025.
- [17] "8.6. Residual Networks (ResNet) and ResNeXt," Dive into Deep Learning, [Online]. Available: https://d2l.ai/chapter_convolutional-modern/resnet.html. [Accessed 15 August 2026].
- [18] C. B. Delahunt, C. Mehanian, D. E. Shea and M. P. Horning, "Beyond validation loss: Clinically-tailored optimization metrics improve a model's clinical performance," *arXiv*, 2026.
- [19] "Writing your own callbacks," [Online]. Available: https://keras.io/guides/writing_your_own_callbacks/. [Accessed 15 August 2026].
- [20] "Region Proposal Object Detection with OpenCV, Keras, and TensorFlow - GeeksforGeeks," [Online]. Available: <https://www.geeksforgeeks.org/deep-learning/region-proposal-object-detection-with-opencv-keras-and-tensorflow/>. [Accessed 15 August 2026].
- [21] "M. Ghorbani, martin-ghorbani/IoU-from-Scratch," [Online]. Available: <https://github.com/martin-ghorbani/IoU-from-Scratch>. [Accessed 15 August 2026].